台灣自編大學教科書

地圖建構與同步定位 (SLAM)

Simultaneous Localization and Mapping



大學教科書

第七篇

第25–27章

微型機器人、無人機與邊緣 AI SLAM

李世淵 (機器人叫獸)

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第七篇 微型機器人、無人機與邊緣 AI SLAM

第 25 章

NanoSLAM 與低功耗 SLAM

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

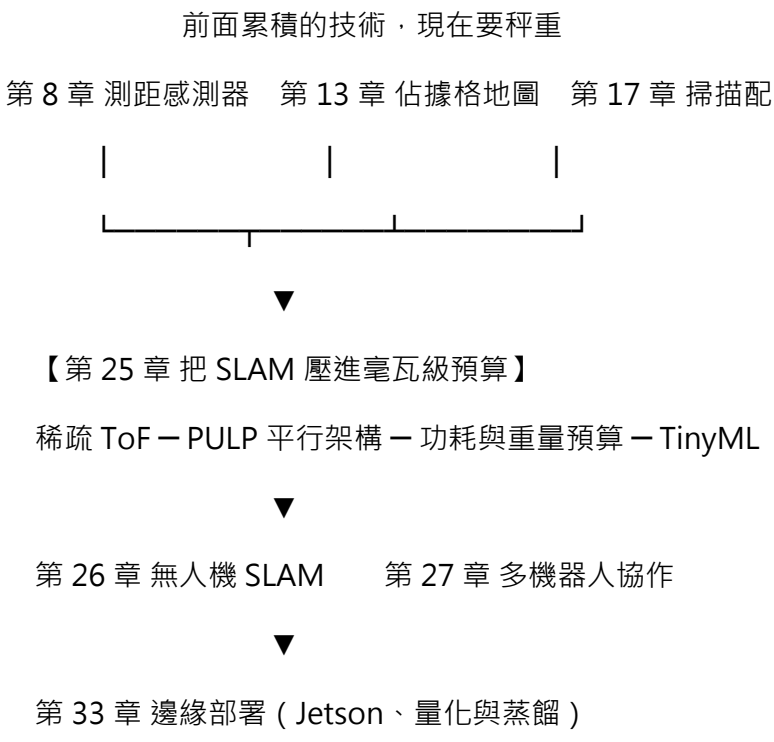
本章在全書的位置

第六篇的視覺 SLAM 已經相當成熟，但它有一個沒說出口的前提：機器人身上有一台夠力的電腦，而且插得到電或背得動電池。本章把這個前提拿掉，看看會剩下什麼。

第七篇的主角是極限受限的平台——一台 27 公克的微型無人機、一顆功耗 66 毫瓦的晶片、七分鐘的電池。在這個預算下，第 13 章的佔據格地圖、第 17 章的掃描配準、第 22 章的關鍵幀管理，每一項都必須重新秤過。

本章的三張圖就在秤這些東西：圖 25-1 解釋為什麼多核慢跑比單核快跑省電，圖 25-3 問「這麼稀疏的量測還能定位建圖嗎」，以及 GAP9 的能效究竟領先多少；圖 25-4 則算出一件可能讓同學意外的事——運算模組的「重量」比它的「功耗」更吃續航。

圖 25-0　第 25 章的連結地圖



學習目標

前六篇的所有系統，都預設機器人有足夠的運算資源與電力——ORB-SLAM3 需要一顆完整的處理器、光達 SLAM 需要數十瓦的運算平台。但如果機器人只有 44 公克重、掌心大小、

電力預算只有毫瓦等級呢？本章展示這個領域最令人驚嘆的工程成就，也開啟第七篇的極限挑戰。學完本章之後，同學應能夠達成下列目標：

1. 說明微型機器人執行 SLAM 的三大限制與傳統解法的問題。
2. 說明 NanoSLAM 的設計目標與五項核心貢獻。
3. 說明 RISC-V 與平行超低功耗運算典範如何支撐毫瓦級 SLAM。
4. 說明毫瓦級 SLAM 的系統設計取捨與各模組的功耗分配。
5. 舉例說明微型無人機、群體機器人與 AR 眼鏡的應用場景。
6. 說明邊緣 AI 與 TinyML 對 SLAM 發展的影響。
7. 規劃一個高中程度的微型無人機地圖建構模擬專題。

核心概念

中文名詞	英文名詞	概念說明
微型無人機	Nano-UAV / Nano-drone	掌心大小、數十公克重的超小型飛行器。
機載運算	Onboard Computation	所有運算都在機器人自身完成，不依賴外部設施。
運算卸載	Computation Offload	把運算送到外部電腦執行，受限於通訊頻寬與延遲。
平行超低功耗	PULP	以多核心平行處理換取能效的運算典範。
系統單晶片	SoC	把處理器、記憶體與周邊整合在單一晶片上。
開放指令集架構	RISC-V	開放且可擴充的處理器指令集架構。
飛時測距矩陣	ToF Matrix Sensor	一次量測多個方向距離的小型深度感測器。
掃描匹配	Scan Matching	比對前後量測以估計相對運動（第 13、17 章）。

中文名詞	英文名詞	概念說明
位姿圖優化	Pose Graph Optimization	以圖優化修正累積誤差（第 12 章）。
動態頻率調節	Dynamic Frequency Scaling	依負載即時調整時脈與電壓以省電。
深度睡眠	Deep Sleep	閒置時把功耗降到極低的省電狀態。
邊緣人工智慧	Edge AI	在裝置端而非雲端執行 AI 推論。
微型機器學習	TinyML	在微控制器等極受限硬體上執行機器學習。

25.1 為什麼微型機器人很難跑 SLAM

第七篇的第一個問題是：為什麼把 SLAM 裝進微型機器人這麼困難？答案不只是「晶片比較慢」這麼簡單，而是三重限制的疊加。

三大限制

- 運算能力：SLAM 的前端（特徵提取、掃描匹配）與後端（圖優化、BA）都極耗運算。第 22、23 章的 ORB-SLAM 系列需要完整的多核心處理器，微控制器等級的晶片根本跑不動。
- 記憶體：第 13 章算過，一張 20×20 公尺、5 公分解析度的 2D 地圖就有 16 萬格；第 14 章的 Gmapping 還要乘上粒子數。而微型平台的記憶體可能只有數百 KB 至數 MB。
- 電力：這是最嚴苛的限制。微型無人機的電池能量極少，而飛行本身就消耗絕大部分電力。若運算再耗掉數瓦，飛行時間會從數分鐘掉到數十秒——系統形同無用。

傳統解法：運算卸載，以及它的問題

由於執行 SLAM 演算法所需的龐大運算負載與記憶體需求，目前完全機載的建圖只能在能搭載高瓦數處理器的機器人平台上實現；正因如此，口袋大小、硬體受限的機器人只能把 SLAM 的執行卸載到外部設施。

這個做法有明顯的代價：

- 依賴基礎設施：必須有無線網路與一台外部電腦，機器人無法真正自主。這在災害現場、地下坑道、野外環境根本不可行。
- 通訊延遲：影像或感測資料傳出去、指令傳回來，延遲可能達數十至數百毫秒。對高速飛行的無人機而言，這個延遲可能導致撞牆。
- 頻寬與功耗：持續傳輸大量資料本身也耗電，而且多台機器人同時運作時頻寬會成為瓶頸（第 27 章的多機器人議題）。

- 可靠度：網路一斷，機器人就失去導航能力。

本章的核心問題

因此本章要回答的問題是：能不能讓 SLAM 完全在機載完成，而且功耗低到微型無人機負擔得起？

答案是肯定的，而且已經被實現了。這需要三個層面的同步突破：演算法必須輕量化、硬體必須高能效、感測器必須低功耗。接下來的幾節，就是這三者如何協同的故事。

25.2 NanoSLAM 的設計目標

NanoSLAM 由蘇黎世聯邦理工學院（ETH Zürich）與波隆那大學的團隊於 2023 年提出，是第 7、25 章反覆提及的代表性工作。它的目標明確而大膽：讓微型無人機完全靠自己執行 SLAM。

核心宣言

NanoSLAM 是一套輕量化且最佳化的端到端 SLAM 方法，專為在僅 87.9 毫瓦的功耗預算下，於公分尺寸的機器人上運作而設計。研究團隊在真實場景中展示其建圖能力，並把 NanoSLAM 部署在一台重 44 公克、搭載名為 GAP9 的新型商用 RISC-V 低功耗平行處理器的微型無人機上。

讓同學對這些數字有感：87.9 毫瓦大約是一顆 LED 指示燈的功耗等級；44 公克比一顆雞蛋還輕。在這樣的條件下執行完整的 SLAM——這是本章最令人驚嘆之處。

五項核心貢獻

NanoSLAM 論文明確列出五項貢獻，它們構成了本章的技術骨架：

1. 平行化的最佳化實作：針對演算法進行最佳化的平行實作，並評估其執行時間與記憶體需求。

2. 機載 ICP：對掃描匹配的標竿演算法 ICP（第 13、17 章）進行平行實作與評估，能在機載以 55 毫秒完成。
3. 感測擴充板：為商用的 Crazyflie 2.1 微型無人機設計客製化的外掛伴隨板，以 4 顆 ToF 矩陣感測器擴充無人機的感測能力，使其能執行掃描匹配與自主導航。
4. 通訊協定：設計一套通訊協定，協調無人機原廠微控制器與 GAP9 系統單晶片之間的整合與資料交換，規範如何儲存圖結構、交換圖的位姿、新增邊，以及執行圖優化。
5. 實地驗證：廣泛的實地實驗評估，證明系統的閉環建圖功能——運用 NanoSLAM 達成軌跡誤差最多降低 67%，以及 4.5 公分的建圖精度。

關鍵成果數據

這個演算法設計上善用 RISC-V 處理核心的平行能力，能以 4.5 公分的精度建立一般環境的地圖，端到端執行時間小於 250 毫秒。

把這些數字與前面章節對照，更能體會其價值：第 23 章 ORB-SLAM3 在 EuRoC 上的 3.5 公分精度，是在完整的電腦上跑出來的；NanoSLAM 的 4.5 公分，則是在毫瓦級的微型平台上達成。研究團隊指出，藉由結合新型低功耗深度感測器與 GAP9 的平行能力，系統達成了與更大、運算能力更強的無人機（通常稱為微型飛行器或標準尺寸無人機）相當的室內建圖精度。

25.3 RISC-V GAP9 與低功耗平行運算

演算法再輕量，也需要合適的硬體才能實現。本節介紹讓毫瓦級 SLAM 成為可能的關鍵硬體。

微控制器設計的新趨勢

近年微控制器設計的趨勢強調平行處理、硬體加速器與能源效率。GreenWaves Technologies 的 GAP9 系統單晶片正是這些趨勢的體現，適用於微型無人機的專門應用；它以 RISC-V 核心提供先進的平行能力、電源最佳化與感測器整合，賦予微型無人機即時邊

緣運算、更長的飛行時間與更強的資料處理能力。

PULP：平行超低功耗運算典範

GAP9 建立在平行超低功耗（Parallel Ultra-Low-Power,PULP）運算典範之上，體積小，功耗低於 180 毫瓦。

PULP 的核心思想值得同學理解，因為它與直覺相反：與其用一顆跑得很快的核心，不如用多顆跑得慢的核心平行工作。原因在於晶片的功耗大致與電壓平方及頻率成正比——把頻率降低一半、用兩顆核心平行處理，總運算量相同，但功耗可能降到原本的一半以下。這是「以平行換能效」的工程智慧。

晶片規格對照

把微型無人機常見的幾種晶片放在一起比較，能清楚看出 GAP9 的定位：

晶片（架構）	CPU 核心數	最高頻率	功耗	定位
STM32(ARM)	1	168 MHz	116 mW	飛控與狀態估計
ESP32(Xtensa)	2	240 MHz	300 mW	通訊與輕量運算
GAP8(RISC-V)	8 + 1	175 MHz	96 mW	前一代 AI 加速
GAP9(RISC-V)	9 + 1	370 MHz	66 mW	本章的主角

表 25-1　微型無人機常見系統單晶片比較

這張表格說明了三件事：第一，GAP9 的核心數遠多於傳統微控制器，體現 PULP 的平行思想；第二，即使核心多、頻率高，它的功耗反而更低，能效優勢明顯；第三，飛控用的 STM32 與運算用的 GAP9 是分工的關係——這正是 25.2 節第四項貢獻（通訊協定）存在的原因。

為什麼是 RISC-V

RISC-V 是一套開放的指令集架構，任何人都能自由使用與擴充，不需授權費。這對研究與新創極為有利：

- 可客製化：能為特定運算（如 SLAM 中的矩陣運算）加入專用指令，大幅提升能效。
- 無授權成本：對學術研究與小量產品的成本結構影響顯著。
- 生態成長快：近年 RISC-V 在邊緣 AI 與物聯網領域快速崛起，工具鏈日趨成熟。

對本書讀者而言，認識 RISC-V 的價值在於：它代表了「運算硬體不再由少數廠商壟斷」的趨勢，也是台灣半導體與機器人產業值得關注的方向。

進階功能：動態調節與深度睡眠

GAP9 支援執行時期調整時脈頻率（最高 370 MHz）與電壓，以及自動時脈閘控；閒置功耗可降至 45 微瓦。

這意味著系統能「該快則快、該省則省」：執行 SLAM 時全速運轉，飛行平穩無需建圖時降頻甚至進入深度睡眠。45 微瓦的閒置功耗幾乎可以忽略不計——這種細緻的電源管理，正是延長飛行時間的關鍵。

量一量：多核慢跑真的比較省電嗎

25.3 節說「與其用一顆跑得很快的核心，不如用多顆跑得慢的核心平行工作」，理由是「功耗大致與電壓平方及頻率成正比」。這個推論值得實際算一遍，因為它還有一個同樣重要的下半段：核心不是越多越好。

模型很簡單。要達成固定的總吞吐量，用 N 顆核心時每顆只需跑 1 / N 的頻率。動態功耗是 $N \times C \times V^2 \times f$，而在近閾值區電壓大致隨頻率線性下降，但有一個下限（電晶體不能低於閾值電壓）。此外每顆核心都會貢獻漏電流，與頻率無關。

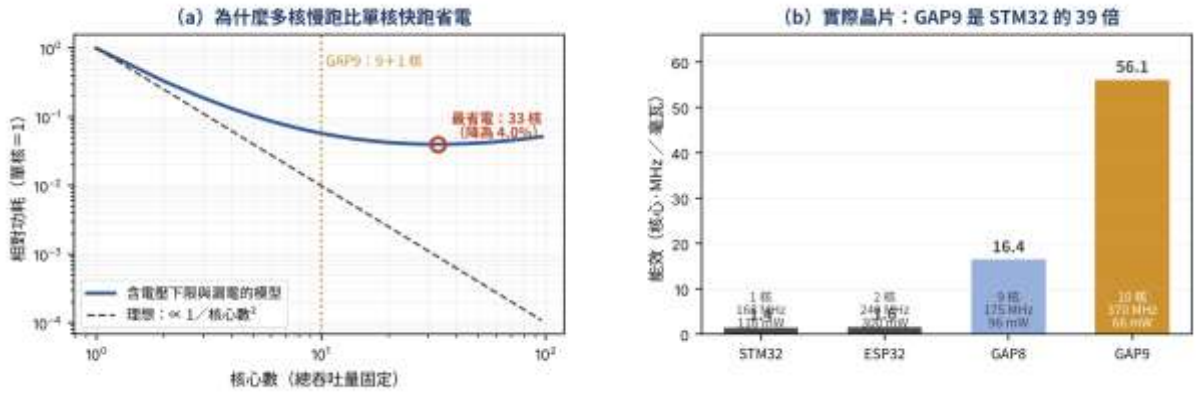


圖 25-1　PULP 的能效原理與實際晶片的對照

（a）是模型的結果，兩軸都是對數刻度。灰色虛線是理想情況——若電壓能隨頻率無限下降，功耗會正比於核心數的平方分之一，也就是核心加倍、功耗降為四分之一。藍線是加入電壓下限與漏電之後的實際曲線：前段貼著理想線走，從 1 核到 8 核功耗降到 6.8%，之後逐漸偏離，在 32 核左右觸底（降到 4.0%），再往上反而因為漏電而回升。

這個最低點解釋了一個同學可能沒想過的問題：既然多核省電，為什麼 GAP9 只放 9＋1 顆核心，不放九十顆？因為過了最佳點之後，每多一顆核心帶來的漏電超過它省下的動態功耗。實際晶片的最佳核心數還要再考慮面積成本、記憶體頻寬與平行化效率——真實的工作負載很少能完美切成九十份。

（b）用表 25-1 的真實晶片數據驗證這個原理。把「核心數乘以頻率」當作吞吐量的粗略指標，除以功耗，得到的能效是：STM32 每毫瓦 1.4、ESP32 1.6、GAP8 16.4、GAP9 56.1。GAP9 是 STM32 的 39 倍、ESP32 的 35 倍。

這個倍數不是全部來自平行度。GAP8 到 GAP9 的能效從 16.4 跳到 56.1（3.4 倍），而核心數只從 9 增加到 10——差距主要來自製程、記憶體階層與專用加速器的改進。換句話說，PULP 的平行思想提供了第一個數量級，後續的硬體最佳化又提供了一個數量級。兩者疊加，才讓 SLAM 從瓦級降到毫瓦級。

25.4　毫瓦級 SLAM 的系統設計

本節說明 NanoSLAM 如何把第 13、17 章的技術，壓縮進毫瓦級的預算。

感測器選擇：ToF 矩陣而非光達或相機

第八章介紹的旋轉式 2D 光達雖然好用，但對微型無人機而言太重、太耗電。相機則需要大量運算才能提取深度（第 9 章）。NanoSLAM 的選擇是 ToF 矩陣感測器——一種小型的飛時測距晶片，一次能量測多個方向的距離。

- 優點：體積僅數公釐、功耗極低、直接輸出距離（不需運算）、無移動部件。
- 配置：如 25.2 節所述，以 4 顆 ToF 矩陣感測器分別朝向四個方向，合起來提供近似 360 度的距離量測——這相當於一個極簡版的 2D 光達。
- 限制：解析度與量測範圍遠低於真正的光達，因此演算法必須能在稀疏的量測下運作。

演算法管線

NanoSLAM 的整體流程，是同學已經熟悉的架構，只是每個模組都被極致輕量化：

圖 25-2 NanoSLAM 的系統管線

【感測】4 顆 ToF 矩陣感測器 → 近似 360 度的稀疏距離量測

↓

【前端】平行化 ICP 掃描匹配（第 13、17 章）

估出相鄰時刻的相對位姿 → 機載 55 毫秒完成

↓

【圖建構】把位姿存為節點、掃描匹配結果存為邊（第 12 章的位姿圖）

↓

【閉環偵測】辨識重訪位置 → 新增閉環邊

↓

【後端】平行化位姿圖優化（第 12 章）

修正累積漂移 → 軌跡誤差最多降低 67%

↓

【輸出】機載完成的地圖與軌跡 端到端 < 250 毫秒，87.9 毫瓦

三個關鍵的輕量化策略

1. 平行化而非簡化：NanoSLAM 沒有把 ICP 或圖優化「改成簡單的近似版」，而是把標準演算法平行化到多核心上。這保住了精度，又善用了 PULP 硬體的特性。
2. 稀疏而非稠密：用 ToF 矩陣提供的稀疏量測取代稠密點雲，大幅減少資料量與運算量——這與第 18 章 LOAM「不需要用上所有的點」的思想一脈相承。
3. 分工而非全包：飛控與狀態估計由原廠的 STM32 負責，SLAM 運算由 GAP9 負責，兩者以專門的通訊協定協調。各司其職，避免單一晶片超載。

功耗分配的啟示

把相關數據放在一起，能看出系統設計的取捨：

任務	功耗 / 延遲	說明
NanoSLAM 建圖	約 87.9 mW，延遲 < 250 ms	本章主角
蒙地卡羅定位	平均 23 mW	已知地圖時的定位（第 11 章）
YOLO 物件偵測	低於 100 mW，最短 17 ms	邊緣 AI 推論（第 24、32 章）
深度睡眠	45 µW	閒置時的極低功耗

表 25-2　微型平台上各項任務的功耗表現

系統維持在低於 100 毫瓦的功耗範圍——尤其是對運算密集的任務而言——對延長飛行時間與支援複雜的即時任務負載至關重要。

值得注意的是：定位（23 mW）遠比建圖（87.9 mW）省電。這印證了第 14 章案例 14-3 的觀察——首次建圖與後續定位是不同的任務。對微型機器人而言，這意味著一個實用策略：先建好地圖（可以慢一點、耗電一點），之後的任務只需低功耗的定位即可。

量一量：這麼稀疏的量測，還能定位嗎

25.4 節說 ToF 矩陣「解析度與量測範圍遠低於真正的光達，因此演算法必須能在稀疏的量測下運作」。這句話留下兩個沒回答的問題：稀疏到什麼程度還能用？以及如果要取捨，該保射線數還是保量程？本小節把兩者分開來量。

實驗設定是一間 10×7 公尺、含六件家具的室內場景。機器人沿橢圓軌跡移動，每個位置用似然場掃描配準估計自己的位姿（第 17 章的方法），與真實位姿比較。兩種感測器配置：4 顆 ToF 矩陣（各 45 度視野、合計約 180 度），以及一台旋轉式光達（360 度、12 公尺）。

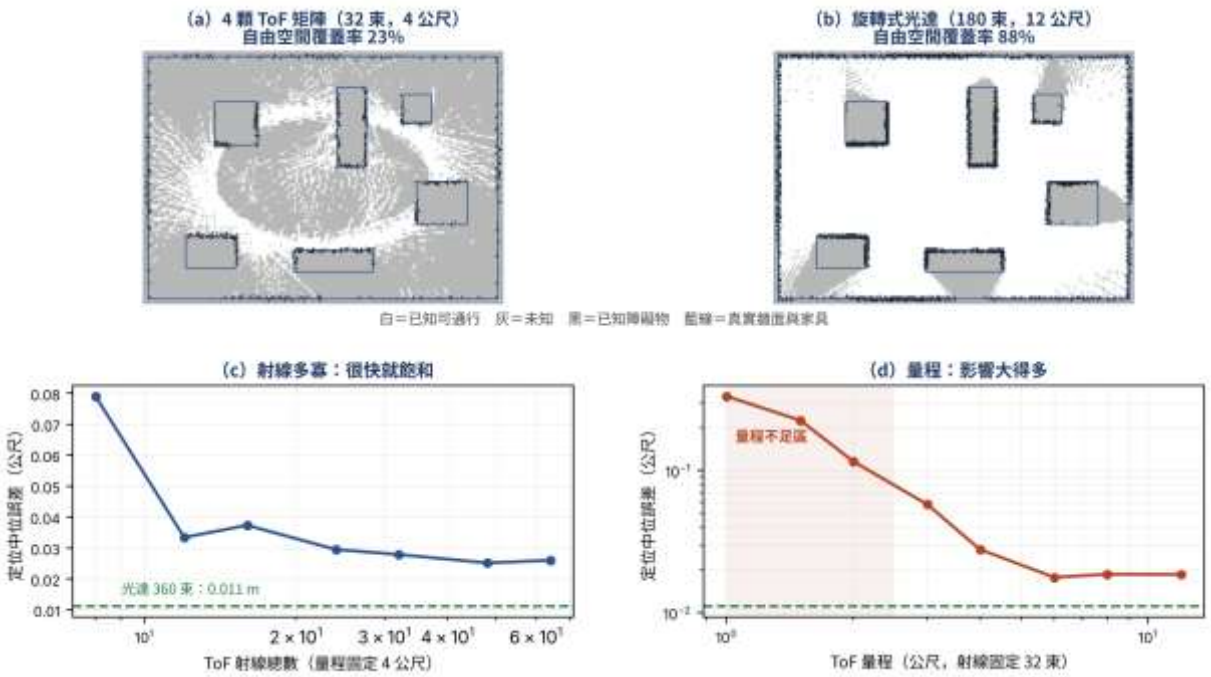


圖 25-3　稀疏 ToF 與旋轉式光達的建圖與定位

（a）與（b）是兩者建出來的地圖。光達把房間掃得乾乾淨淨，自由空間覆蓋率 88%，六件家具的輪廓都清楚。ToF 的地圖則明顯殘缺：覆蓋率只有 23%，白色的可通行區域呈放射狀，大片區域仍是未知的灰色。原因有兩個——4 顆感測器只涵蓋約一半的方位，而且量程只有 4 公尺，掃不到遠處的牆。

（c）回答第一個問題：射線數。只有 8 束時定位誤差是 0.079 公尺，增加到 12 束就降到 0.033 公尺，之後幾乎走平——32 束是 0.028、64 束是 0.026 公尺。也就是說，射線從 12 束加到 64 束（五倍以上），精度只改善兩成。這是一個相當重要的結論：對定位而言，十幾束量測就已經接近飽和。

（d）回答第二個問題：量程，而答案完全不同。量程 1 公尺時定位誤差是 0.334 公尺（根本不能用），2 公尺時 0.116、4 公尺時 0.028、6 公尺以上才走平在 0.018 公尺。從 1 公尺到 4 公尺，誤差改善了十二倍。把（c）與（d）並排看，結論很清楚：在微型平台上挑 ToF 感測器時，量程比區段數重要得多。

最後把 ToF 與光達放在一起：4×8 的 ToF 配置在 4 公尺量程下定位誤差是 0.028 公尺，光達是 0.011 公尺——只差 2.5 倍，而射線數只有十一分之一。這正是毫瓦級 SLAM 得以成立

的關鍵：定位這件事，對量測密度的需求遠低於直覺。

定位夠用，不代表地圖夠用

這張圖裡有一個必須講清楚的落差：ToF 的定位精度只比光達差 2.5 倍，但地圖覆蓋率只有光達的四分之一（23% 對 88%）。

原因是兩件事需要的量測量不同。定位是把當前這一幀對上已知的結構，少數幾個正確的距離量測就能把六個自由度釘住；建圖卻要把每一塊空間都實際掃過一遍，沒有捷徑。

這也解釋了真實的毫瓦級系統為什麼多半只建「夠用的粗地圖」——不是演算法不行，而是感測器的總量測量就那麼多。同學在規劃微型平台的專題時，應該把目標訂成「能回到起點、能避開障礙」，而不是「建出和光達一樣漂亮的地圖」。

25.5 微型無人機、Swarm Robots、AR 眼鏡應用

毫瓦級 SLAM 開啟了哪些原本不可能的應用？本節說明三個代表性方向。

微型無人機：安全與可及性

微型無人機能放進手掌，適合室內環境，且與人互動時很安全；然而它們常因缺乏足夠的感測與運算硬體，而無法在複雜任務中提供所需的效能。

「安全」這一點特別值得強調。一台數公斤的標準無人機若失控撞到人，可能造成嚴重傷害；而 44 公克的微型無人機幾乎不具危險性。這使得它們能在教室、辦公室、醫院等有人的空間中運作——這是大型無人機做不到的。

- 室內探索與測繪：在人無法或不宜進入的空間（狹窄管道、災後建築、天花板夾層）進行探索與建圖。
- 農業巡檢：已有研究以多台自主微型無人機視覺偵測害蟲，再由單一較慢的重型載具前往偵測到的爆發點施藥；為因應微型無人機的極端限制（如低解析度感測器與低於

100 毫瓦的運算功耗預算），團隊設計並最佳化了一個微型的影像式卷積神經網路用於害蟲偵測。

- 教育與研究：微型無人機價格相對親民、安全性高，是極佳的教學平台（25.7 節）。

群體機器人（Swarm Robots）

當單台機器人的成本與體積都降低，「用很多台」就成為可行的策略。群體機器人的價值在於：

- 覆蓋效率：多台分頭探索，完成大範圍任務的時間大幅縮短（第 24 章案例 24-3 的多機器人協作探索）。
- 容錯性：單台故障不影響整體任務——這是單一昂貴機器人無法比擬的優勢。
- 分散式感知：從多個視角同時觀測，能建立更完整的環境模型。

但群體機器人有一個關鍵前提：每台機器人必須能自主定位。若全部依賴外部設施或中央電腦，通訊頻寬會隨機器人數量成長而崩潰（25.1 節提過）。機載 SLAM 因此是群體機器人可規模化的必要條件——這正是 NanoSLAM 這類技術的戰略價值。第 27 章會深入多機器人 SLAM 的協作議題。

AR 眼鏡與穿戴裝置

第 20、23 章談過 AR 需要視覺慣性 SLAM，但 AR 眼鏡的限制比手機更嚴苛：必須輕到能長時間配戴、電池必須小、且不能發熱燙臉。

這與微型無人機的限制驚人地相似——都是「極小的功耗預算下要跑 SLAM」。因此本章的技術路線（專用低功耗晶片、平行超低功耗運算、輕量化演算法）也直接適用於穿戴裝置。

這個跨領域的共通性，是本章一個重要的觀念：低功耗 SLAM 不只是微型無人機的專門技術，它是所有「受限平台」的共同需求。掌握這條路線，同學就掌握了通往 AR 眼鏡、智

慧手錶、微型感測節點等眾多應用的鑰匙。

量一量：運算的代價，真的是功耗嗎

25.1 節說微型機器人的困難在於「運算、功耗與重量三重限制」。三者當中哪一個最致命？直覺上會說功耗，但算過之後會發現答案是重量——而且差距很大。

計算基礎是一台 27 公克等級的微型無人機：電池約 0.89 瓦時、懸停功耗約 7.6 瓦、純懸停續航約 7.0 分鐘。依動量理論，懸停功耗大致與質量的 1.5 次方成正比，所以多掛一塊板子不只是多耗那塊板子的電，還會讓四個馬達全部更費力。

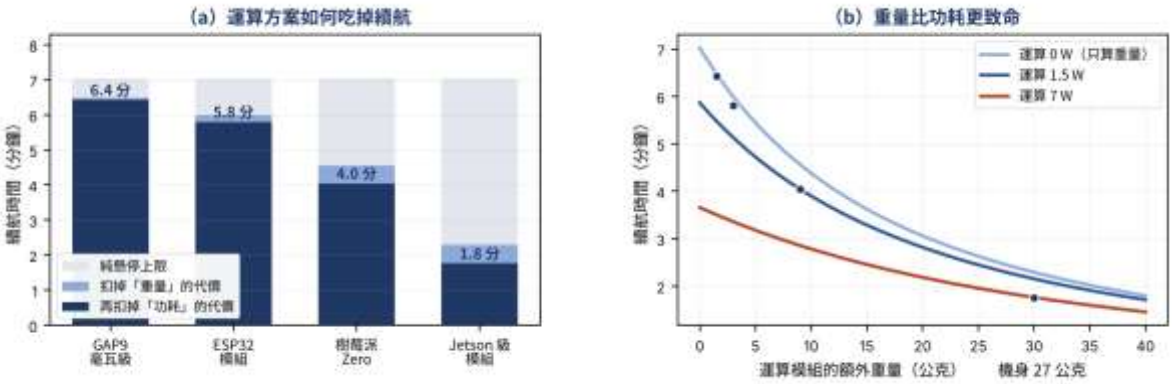


圖 25-4　運算方案的重量與功耗如何吃掉續航

（a）把四種運算方案的續航拆成兩段損失。以 GAP9 為例：它只有 1.5 公克、66 毫瓦，重量讓續航從 7.03 分降到 6.48 分，功耗再降到 6.43 分——總共只損失 8.5%。

Jetson 等級的模組則是另一個極端：30 公克、7 瓦。重量把續航從 7.03 分壓到 2.29 分，功耗再壓到 1.76 分。請注意這兩段的比例——重量造成 4.74 分的損失，功耗只造成 0.53 分，前者是後者的 9 倍。四種方案的這個比例分別是 10.6、5.3、4.7 與 9.0 倍，沒有一個例外。

（b）把重量單獨拉出來看。即使運算功耗是 0 瓦（想像一塊不耗電的板子），光是掛上 30 公克就會讓續航從 7.0 分掉到 2.3 分。三條曲線之間的垂直距離（也就是功耗的影響）遠小於曲線本身的下降幅度。

這個結論對專題選型有直接的意義。同學在為微型平台挑運算模組時，第一個該問的不是「它耗幾瓦」，而是「它多重」。一塊省電但笨重的板子，可能比一塊耗電但輕巧的板子更傷續航。這也是 25.2 節為什麼要強調「毫瓦級」與「數公克」必須同時成立——兩者缺一，微型 SLAM 就不成立。

> **這條規則在什麼時候會反過來**
>
> 重量優先的結論，前提是「懸停功耗遠大於運算功耗」。對 27 公克的無人機而言，7.6 瓦對 0.066 瓦，相差一百倍以上，所以重量完全主導。
>
> 但換成地面機器人就不一樣了。一台 5 公斤的輪型機器人，移動功耗可能只有 10 瓦，而多掛 100 公克幾乎不影響能耗——此時運算功耗反而成為主要考量。
>
> 所以這不是一條普遍規則，而是一條「飛行平台的規則」。同學在套用前，先算一下自己平台的「移動功耗對運算功耗」比值，以及「額外重量對自身重量」比值。

25.6 Edge AI 與 TinyML 對 SLAM 的影響

本節把視野從 SLAM 擴大到整個邊緣智慧的趨勢，並說明它與本書後續章節的連結。

邊緣 AI 的意義

邊緣 AI 指的是在裝置端而非雲端執行 AI 推論。它的價值與 25.1 節討論機載 SLAM 的理由完全一致：不依賴網路、低延遲、隱私保護、以及可規模化。

TinyML 則是邊緣 AI 的極端版本——在微控制器等記憶體只有數百 KB 的硬體上執行機器學習。這聽起來不可思議，但透過模型量化、剪枝、知識蒸餾等技術，已經能在極小的硬體上執行實用的推論。

與 SLAM 的三種結合

1. 感知增強：在微型平台上執行物件偵測。相關評測顯示，推論任務如 YOLO 物件偵測能在低於 100 毫瓦下完成，最短推論時間為 17 毫秒、約 1.59 毫焦耳。這讓微型機器人也能做語意感知（第 24 章）。
2. 學習取代運算：用小型神經網路直接估計某些量（如光流、深度、甚至位姿），取代傳統的幾何運算。若模型夠小，推論可能比傳統演算法更省電。
3. 線上學習：更前沿的方向是讓機器人在飛行中自我學習調適。已有研究探討在極低功耗預算下於微型無人機上進行裝置端自監督學習——這代表機器人不只是執行預先訓練好的模型，還能持續適應新環境。

硬體與演算法的共同設計

本章最重要的方法論啟示是：在極限受限的條件下，硬體與演算法不能分開設計。

NanoSLAM 之所以成功，不是因為演算法特別聰明，也不是因為晶片特別快，而是因為兩者互相配合：演算法為 RISC-V 多核心的平行架構量身打造，硬體則提供了演算法所需的能效與整合度。這種「硬體軟體共同設計」（Hardware-Software Co-design）的思維，是邊緣運算領域的核心能力。

對智慧機器人學程的同學而言，這是一個重要的提醒：除了學會使用現成的演算法，理解底層硬體的特性（平行度、記憶體階層、功耗行為）同樣重要。這兩者的交會處，正是台灣半導體優勢與機器人應用能夠結合的地方。

與後續章節的連結

- 第 26 章：微型無人機的視覺慣性導航，延續本章的平台限制討論。
- 第 27 章：多機器人與雲端邊緣協作，呼應 25.5 節的群體機器人。
- 第 32–34 章：語意 SLAM、模擬訓練與邊緣 AI 部署（Jetson、NPU，第 33 章），以及 VLM 語言導航（第 34 章），是本章邊緣運算討論的延伸。

25.7 高中專題：微型無人機地圖建構模擬

本章的技術聽起來遙不可及，但它其實非常適合作為高中專題——因為核心觀念可以用模擬完整體驗，不需昂貴硬體。

為什麼適合高中

- 觀念直觀：「用四個方向的距離感測器建地圖」比視覺 SLAM 的特徵與幾何容易理解得多。
- 可完全模擬：用 Python 就能寫出一個微型無人機的 2D 模擬，不需飛行器。
- 連結有感：微型無人機、群體機器人、AR 眼鏡都是學生感興趣的主題。
- 層次分明：從最簡單的「感測與畫圖」，到「加入漂移」，再到「閉環修正」，難度可調。

專題設計：四個階段

1. 階段一——建立模擬環境：用 Python 畫出一個 2D 房間（牆與障礙），放入一台虛擬的微型無人機，實作四個方向的距離感測（從無人機位置往四個方向找到最近的牆）。
2. 階段二——理想建圖：讓無人機沿預定路線飛行，每一步用四個方向的距離量測，把偵測到的牆點畫在地圖上。此階段假設位置完全準確，學生會看到一張漂亮的房間地圖逐漸浮現。
3. 階段三——加入漂移：在無人機的位置估計中加入小小的隨機誤差（模擬里程計漂移，第 1、7 章），重新建圖。學生會親眼看到地圖開始扭曲、牆線重影——這是第 13.7 節地圖病徵的具體重現。
4. 階段四——閉環修正：讓無人機繞一圈回到起點，加入「終點應該等於起點」的約束，用簡化的方式把誤差均攤回整條軌跡（第 12 章實作 12-1 的紙上方法可直接套用），觀察地圖被「拉正」的效果。

延伸方向

- 功耗預算計算：給定電池容量與各模組的功耗（參考表 25-2），計算飛行時間，並實驗「若把建圖頻率降低一半，能飛多久？」——這是很好的工程權衡練習。
- 群體模擬：加入多台虛擬無人機分頭探索，比較完成全區覆蓋所需的時間（呼應 25.5 節與第 27 章）。
- 感測器數量實驗：比較用 4 個、8 個、16 個方向的距離感測，對地圖品質的影響——這對應真實系統中「感測器數量與功耗、重量」的取捨。

教學建議

這個專題的教學價值，在於它讓學生在最簡化的情境中，完整走過 SLAM 的核心邏輯：感測 → 建圖 → 發現漂移 → 閉環修正。這正是本書第一篇到第四篇的縮影。

建議教師先讓學生做出階段二的「完美地圖」建立成就感，再刻意加入漂移製造「破壞」——那個「原本漂亮的地圖忽然歪掉」的瞬間，會讓學生真切理解為什麼 SLAM 是一個困難的問題。這種先建立、再打破、最後修復的教學節奏，比直接講解理論有效得多。

至此，同學已經看到 SLAM 在極限受限平台上的實現。下一章我們聚焦無人機本身的導航挑戰：GPS 拒止環境如何飛行？高速運動與震動如何處理？視覺慣性里程計在無人機上如何運作？第 26 章將把第七篇的討論帶向飛行的核心。

25.8 毫瓦級之後：2025–2026 年的三個進展

NanoSLAM 證明了毫瓦級 SLAM 可行。這幾年的進展主要在三個方向，每一個都值得同學知道它的代價。

一、把完整的視覺慣性里程計搬上 RISC-V

2026 年的一項工作把完整的單目視覺慣性管線部署到 GAP9 上：ORB 特徵、八點法與 EPnP 搭配 RANSAC、IMU 預積分、以及滑動視窗的位姿圖優化——也就是第 21、23 章的那一整套，塞進 370 MHz 的晶片。

成績是每秒 20 幀、功耗低於 100 毫瓦、記憶體用量不到 1 MB（第一層約 110 KB、第二層約 850 KB）。但代價很明確：它省掉了閉環偵測，在標準的 EuRoC 資料集上平均軌跡誤差約 3.46 公尺，而有閉環的 ORB-SLAM3 約 0.25 公尺——差了一個數量級。作者自己把原因歸給缺少全域修正。

這個對照值得與第 22 章圖 22-2 一起讀。那張圖證明了閉環與共視邊是誤差重新分配的唯一途徑；而微型平台之所以放棄它們，不是因為不重要，而是因為它們需要保留整張地圖與整段軌跡——那正是 1 MB 記憶體裡放不下的東西。

二、感測器往更稀疏、更便宜走

第二個方向是把感測器再簡化。圖 25-3(c) 的結論——十幾束量測就接近飽和——正是這個方向的理論基礎。近年出現的做法包括：用單顆多區段 ToF 搭配旋轉機構取代多顆固定感測器、以及用極低解析度的灰階相機（例如 160×160）做光流與場所辨識。

這類設計的共同邏輯是：與其追求單次量測的品質，不如增加量測的次數與多樣性，再用第 11 章的機率方法把它們融合起來。這與第八、九章「沒有全能的感測器，只有恰當的組合」是同一個信念，只是在毫瓦級的預算下被推到了極致。

三、運算往「該算才算」走

第三個方向不在演算法也不在硬體，而在排程。25.3 節提過 GAP9 的閒置功耗可以降到 45 微瓦，這讓「事件驅動」的運算變得划算：平穩飛行時降頻甚至休眠，只在轉彎、遇到障礙或偵測到定位不確定度上升時才全速運轉。

這個思想與第 19.6 節的地圖維護、第 33.8 節的錯開執行是一脈相承的——都是在問「這一步真的需要現在做嗎」。對續航只有七分鐘的平台而言，省下一半的運算時間，就等於多飛將近半分鐘。

	本章的 NanoSLAM	RISC-V 上的完整 VIO	事件驅動排程

感測器	4 顆 ToF 矩陣	單目相機 + IMU	不限
有無閉環	有（輕量）	無	視配置
記憶體	數百 KB	不到 1 MB	不變
功耗	毫瓦級	低於 100 毫瓦	平均可再降
精度	公分級（小場域）	EuRoC 約 3.46 公尺	不變
主要代價	地圖稀疏	無全域修正	實作複雜度

表 25-3　三個方向的比較

公式或流程

一、功耗與飛行時間的關係

微型無人機的飛行時間，由電池能量與總功耗決定：

$$\text{飛行時間} = \text{電池能量} \;/\; (\text{飛行功耗} + \text{運算功耗} + \text{感測功耗})$$

以 Crazyflie 2.1 為例，搭配 380 mAh 電池時單次飛行可達約 7 分鐘。假設飛行本身的功耗遠大於運算，則：

$$\text{若運算功耗佔總功耗比例為 } r\text{，則飛行時間下降約 } r$$

這條簡單的關係式解釋了為什麼「毫瓦」如此關鍵。若 SLAM 耗電 5 瓦，對一台總功耗約 10 瓦的微型無人機而言，飛行時間會直接砍半；而 87.9 毫瓦僅佔約 1%，幾乎不影響續航。這就是 NanoSLAM 把功耗壓到毫瓦級的工程意義。

二、平行加速的效益上限

把演算法平行化到 N 個核心上，理論加速比受限於「無法平行的部分」：

$$\text{加速比 } S(N) = 1 \;/\; ((1 - p) + p \;/\; N)$$

其中 p 為可平行化的比例。這稱為阿姆達爾定律（Amdahl's Law）。例如 p = 0.9、N

= 8 時，加速比約為 4.7 倍，而非 8 倍。

這條式子對 NanoSLAM 的設計有直接意涵：ICP 的最近點搜尋高度可平行（每個點獨立搜尋），因此 p 很高、平行效益顯著；而圖優化中的稀疏矩陣分解則較難完全平行。理解哪些部分值得平行化，是善用 PULP 架構的關鍵。

三、能效指標

比較不同平台的運算能力時，單看速度不夠，要看「每焦耳能做多少運算」：

$$\text{能效} = \text{運算量} \;/\; \text{能量消耗} \quad （常用單位：GOPS/W 或 Ops/J）$$

這正是 PULP 典範追求的目標（25.3 節）。以本章的資料為例，YOLO 物件偵測最短推論時間 17 毫秒、約 1.59 毫焦耳——用「每次推論消耗多少能量」來衡量，遠比「每秒能跑幾次」更能反映受限平台的實際可用性。

四、低功耗 SLAM 的設計決策流程

圖 25-5 受限平台的 SLAM 設計決策流程

① 功耗預算是多少？（電池容量 ÷ 目標運行時間 - 移動所需功耗）

↓

② 感測器選擇：在預算內能負擔什麼？

- 數瓦 → 3D 光達 / 深度相機
- 數百毫瓦 → 2D 光達 / 單目相機
- 數十毫瓦 → ToF 矩陣感測器（本章方案）

↓

③ 演算法選擇：資料量與運算量能否負擔？

- 稠密方法 → 需要大量記憶體與運算，受限平台不可行
- 稀疏 / 特徵法 → 大幅減量（第 18、20 章的共同思想）

↓

④ 平行化評估：哪些部分可平行？（依阿姆達爾定律估算效益）

↓

⑤ 電源管理：動態調頻、深度睡眠、依任務切換模式

↓

⑥ 分工設計：飛控與運算是否分離到不同晶片？通訊如何協調？

案例

案例 25-1 一顆 LED 的電力：重新理解「運算成本」

本章最值得同學停下來思考的，是那個 87.9 毫瓦的數字。

讓我們做幾個對照：一顆手機 LED 手電筒約消耗數百毫瓦；一台筆記型電腦執行 ORB-

SLAM3 時，處理器功耗可能達到數十瓦——是 NanoSLAM 的數百倍。而 NanoSLAM 在這樣的預算下，達成了 4.5 公分的建圖精度，與更大、運算能力更強的無人機相當。

這個對比顛覆了一個常見的直覺：我們往往認為「更好的結果需要更多的運算」。但 NanoSLAM 證明，透過演算法與硬體的共同設計，可以在功耗降低兩三個數量級的同時，維持相當的精度。

這給同學兩個啟示。第一，效能瓶頸往往不在「算力不夠」，而在「用錯方法」。第 18 章 LOAM「不需要用上所有的點」、第 20 章 ORB「以部分精度換速度」、本章「以平行換能效」——都是同一種思維：找到問題的本質，只做必要的計算。

第二，在資源受限的條件下創新，往往比在資源充裕時更能激發真正的工程智慧。這對台灣的機器人與半導體產業特別有啟發意義：我們或許無法在算力上與大廠競爭，但在「用更少資源做更多事」這條路上，有充分的發揮空間。

案例 25-2　誰負責飛、誰負責想：雙晶片的分工哲學

NanoSLAM 系統中一個容易被忽略、卻極具工程智慧的設計，是它的雙晶片架構：原廠的 STM32 負責飛控與狀態估計，新增的 GAP9 負責 SLAM 運算，兩者以專門設計的通訊協定協調——包括如何儲存圖結構、交換圖的位姿、新增邊，以及執行圖優化。

為什麼不用一顆晶片全包？原因有三：

- 即時性要求不同：飛控迴路必須以極高頻率、極低延遲執行，任何延遲都可能導致墜機；SLAM 則可以慢一點（250 毫秒也能接受）。把兩者放在同一顆晶片上，SLAM 的運算尖峰可能拖累飛控。
- 可靠度隔離：即使 SLAM 當掉，飛控仍能維持飛行安全——這是安全設計的基本原則。
- 各取所長：STM32 適合即時控制，GAP9 適合平行運算。用對的晶片做對的事，整體能效最佳。

同學應該注意到：這個「快速反應層 + 慢速思考層」的分工，與第 3 章的混合式架構、第 15 章 Cartographer 的局部與全域子系統、第 18 章 LOAM 的高頻里程計與低頻建圖，是完全相同的設計模式。

這說明了一件事：好的系統設計原則具有跨層次的普遍性。無論是在演算法層、軟體架構層還是硬體層，「依時間尺度分層、各層各司其職」都是有效的策略。掌握這種模式思維，比記住任何單一系統的細節都更有價值。

案例 25-3　微型無人機的田野應用：從實驗室到農田

微型無人機的價值，在一個具體的農業應用中展現得淋漓盡致。

有研究團隊提出這樣的構想：高效的作物生產需要及早偵測害蟲爆發並及時處理，他們考慮以一支由多台自主微型無人機組成的機隊視覺偵測害蟲，再由單一台較慢的重型載具前往偵測到的爆發點施藥。

這個「輕騎兵偵察、重裝部隊處理」的分工設計極為聰明：微型無人機便宜、安全、可大量部署，適合大範圍快速搜尋；重型載具雖然慢，但能攜帶藥劑執行實際處理。兩者結合，效率遠高於單獨使用任一種。

技術上的挑戰同樣值得注意：為因應微型無人機的極端限制（如低解析度感測器與低於 100 毫瓦的運算功耗預算），團隊設計、微調並最佳化了一個微型的影像式卷積神經網路用於害蟲偵測。這個網路每次推論僅需 0.58 GOps，在他們的資料集上偵測有害昆蟲的平均精度均值達 0.79——相較文獻中表現最佳的網路，平均精度低了 14%，但運算量少了 32 倍。

這個「精度降低 14%、運算量減少 32 倍」的取捨，正是受限平台工程的縮影。這個案例也提示了一個在地化的專題方向：台灣的農業場域（果園、稻田、溫室）同樣面臨病蟲害偵測的需求，而微型無人機的安全性與低成本，使它比大型無人機更適合在密集種植的環境中作業。

實作活動

實作 25-1 微型無人機地圖建構模擬

目的：以純模擬的方式完整體驗低功耗 SLAM 的核心邏輯——從稀疏感測建圖、觀察漂移、到閉環修正，並進行功耗與感測器數量的工程權衡分析。本實作只需 Python，適合高中到大學部各層次。

第一部分：建立模擬環境與理想建圖（全體）

1. 用 Python 定義一個 2D 房間：以線段表示牆與障礙物，並在其中放置一台虛擬微型無人機（以位置與朝向表示）。
2. 實作四方向距離感測：從無人機位置沿前、後、左、右四個方向射出射線，計算與最近牆面的交點距離。這模擬了 25.4 節的 4 顆 ToF 矩陣感測器。
3. 讓無人機沿一條預定路線（例如繞房間一圈）移動，每一步記錄四個距離值，並依第 10 章的座標轉換把偵測到的牆點畫在地圖上。
4. 用 matplotlib 顯示逐步成長的地圖，觀察房間輪廓如何浮現。此階段假設位置完全準確，地圖應該相當漂亮。

第二部分：漂移與閉環（全體）

1. 加入里程計漂移：在無人機每一步的位置與角度估計中加入小的隨機誤差（例如位置誤差標準差 2 公分、角度誤差 0.5 度），重新建圖。
2. 觀察病徵：地圖出現什麼問題？牆線是否重影？繞完一圈後，終點與起點是否對不上？量測這個落差有多大。對照第 13.7 節的三種地圖病徵。
3. 實作簡化的閉環修正：當無人機回到起點附近時，加入「終點應等於起點」的約束，把累積誤差平均分攤到整條軌跡的每一步（第 12 章實作 12-1 的方法），重新繪製地圖。

4. 量化改善：比較修正前後的「終點與起點落差」以及地圖的目視品質，計算誤差降低的百分比，與 NanoSLAM 論文的 67% 對照。

第三部分：工程權衡分析（大學部，本章特色）

1. 感測器數量實驗：把四方向感測改為 8、16、32 個方向，比較地圖品質（牆面完整度、細節解析度）。找出「品質提升趨緩」的轉折點。
2. 功耗估算：假設每個 ToF 感測器耗電 20 毫瓦、每次掃描匹配耗電依感測點數線性成長，計算不同感測器數量下的總功耗。
3. 飛行時間試算：依公式一，假設電池能量固定、飛行本身耗電 5 瓦，計算各種配置下的飛行時間，畫出「感測器數量 vs 地圖品質 vs 飛行時間」的三方權衡圖。
4. 決策練習：若任務要求「至少飛行 5 分鐘且地圖誤差小於 10 公分」，你會選擇哪一種配置？說明理由。

第四部分：延伸探索（進階）

1. 建圖頻率實驗：把 SLAM 執行頻率從每步一次改為每三步、每五步一次，觀察地圖品質下降多少、功耗節省多少。這對應真實系統的動態調頻策略（25.3 節）。
2. 群體模擬：加入第二台、第三台虛擬無人機分頭探索，各自建圖後合併，比較「協作探索」與「單機探索」完成全區覆蓋所需的步數。這呼應 25.5 節與第 27 章。
3. 定位模式：先用一台無人機建好地圖，再讓第二台在已知地圖中只做定位（不建圖），比較兩者的運算量。對照表 25-2 中「定位 23 mW、建圖 87.9 mW」的差異。

討論問題

1. 加入漂移後，地圖出現了哪些病徵？繞一圈的終點落差有多大？這與第 13.7 節的描述是否吻合？
2. 閉環修正把誤差「均攤」到整條軌跡，而不是集中修正最後一步。為什麼均攤比較好？（提示：第 12 章案例）

3. 感測器從 4 個增加到 32 個，地圖品質提升了多少？功耗增加多少？你認為最合適的數量是多少？為什麼？
4. 若你是微型無人機的系統設計者，在「更好的地圖」與「更長的飛行時間」之間，你會如何取捨？這個取捨是否依任務而異？
5. （大學部）依阿姆達爾定律（公式二），若 ICP 有 95% 可平行化，用 GAP9 的 9 個核心能加速多少倍？這與理想的 9 倍差多少？差距從何而來？

延伸挑戰（大學部）

其一：實作一個簡化的位姿圖優化——把模擬中每一步的位姿當作節點、相鄰步之間的運動當作邊、閉環當作額外的邊，用 scipy 的最小平方求解器最佳化整條軌跡（第 12 章實作 12-1 的程式版本），比較它與「平均分攤」的簡易方法在精度上的差異。其二：若學校有 Crazyflie 或類似的微型無人機平台，嘗試讀取其感測器資料並在電腦端執行你的 SLAM 程式，體驗真實資料與模擬的差異——真實的感測雜訊、時間延遲與飛行擾動，會讓你對本章的工程挑戰有更深的體會。

延伸閱讀

1. V. Niculescu et al., "NanoSLAM: Enabling Fully Onboard SLAM for Tiny Robots," IEEE Internet of Things Journal, 2023(arXiv:2309.12008).（本章主要依據，程式碼公開於 GitHub: ETH-PBL/NanoSLAM）
2. H. Müller et al., "GAP9Shield: A 150GOPS AI-capable Ultra-low Power Module for Vision and Ranging Applications on Nano-drones," arXiv:2407.13706, 2024.（微型無人機的感測與運算模組，含功耗實測數據）
3. D. Rossi et al., PULP 平台相關論文與 pulp-platform.org 官方網站.（平行超低功耗運算典範的技術來源）

4. Bitcraze Crazyflie 官方文件（bitcraze.io）.（本章實驗平台的硬體規格、開發環境與社群資源）
5. GreenWaves Technologies GAP9 產品文件.（RISC-V 低功耗處理器的規格與開發工具）
6. P. Warden and D. Situnayake, TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers, O'Reilly, 2019.（TinyML 的入門經典）
7. L. Crupi et al., "Training on the Fly: On-device Self-supervised Learning aboard Nano-drones within 20 mW," arXiv:2408.03168, 2024.（微型無人機上的裝置端學習，25.6 節的前沿方向）

小結

1. 微型機器人執行 SLAM 面臨運算、記憶體與電力三重限制；傳統做法是把運算卸載到外部設施，但這帶來依賴基礎設施、通訊延遲、頻寬功耗與可靠度的問題，使機器人無法真正自主。
2. NanoSLAM 是輕量化且最佳化的端到端 SLAM 方法，專為在僅 87.9 毫瓦的功耗預算下於公分尺寸機器人上運作而設計；它被部署在重 44 公克、搭載 RISC-V 低功耗平行處理器 GAP9 的微型無人機上，達成 4.5 公分建圖精度與小於 250 毫秒的端到端執行時間。
3. GAP9 建立在平行超低功耗（PULP）運算典範上，以多核心平行換取能效，功耗低於 180 毫瓦；它支援動態調整時脈與電壓、自動時脈閘控，閒置功耗可降至 45 微瓦。RISC-V 的開放性使其可客製化且無授權成本。
4. NanoSLAM 以 4 顆 ToF 矩陣感測器取代光達、以平行化 ICP 做掃描匹配（機載 55 毫秒）、以位姿圖優化修正漂移（軌跡誤差最多降低 67%）；三個輕量化策略為平行化而非簡化、稀疏而非稠密、分工而非全包。

5. 微型無人機能放進手掌、適合室內且與人互動安全；機載 SLAM 是群體機器人可規模化的必要條件；而 AR 眼鏡的限制與微型無人機驚人地相似，使低功耗 SLAM 成為所有受限平台的共同需求。
6. 邊緣 AI 與 TinyML 與 SLAM 有三種結合：感知增強（YOLO 可在低於 100 毫瓦、最短 17 毫秒完成推論）、以學習取代運算、以及線上自我學習；核心方法論是硬體與演算法的共同設計。
7. 微型無人機地圖建構模擬極適合高中專題：觀念直觀、可完全模擬、層次分明；建議的教學節奏是先建立完美地圖、再刻意加入漂移製造破壞、最後以閉環修復。

承先啟後

本章把 SLAM 壓進毫瓦級的預算，也量出了三組關鍵數字：定位對量測密度的需求遠低於直覺——12 束 ToF 就接近飽和（0.033 公尺），32 束（0.028 公尺）與光達的 0.011 公尺只差 2.5 倍；但量程的影響大得多，從 1 公尺放寬到 4 公尺可以把誤差改善十二倍。在硬體上，固定吞吐量時多核慢跑可以把功耗降到 4%，但過了約 32 核就會被漏電反噬；實際晶片上 GAP9 的能效是 STM32 的 39 倍。而在平台層面，運算模組的重量對續航的影響是功耗的 4.7 至 10.6 倍——挑板子時第一個該問的是它多重，不是它耗幾瓦。

本章展示了 SLAM 在極限受限平台上的實現，也帶出一個貫穿全書的觀念：效能瓶頸往往不在算力不足，而在方法未經最佳化。下一章我們聚焦無人機導航本身的挑戰——GPS 拒止環境如何飛行？高速運動、震動與影像模糊如何處理？光流、視覺慣性里程計與視覺 SLAM 在無人機上如何分工？第 26 章將把第七篇的討論帶向飛行的核心。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 26 章

無人機 SLAM 與視覺慣性導航

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程 / 高中 AI 機器人社團課程

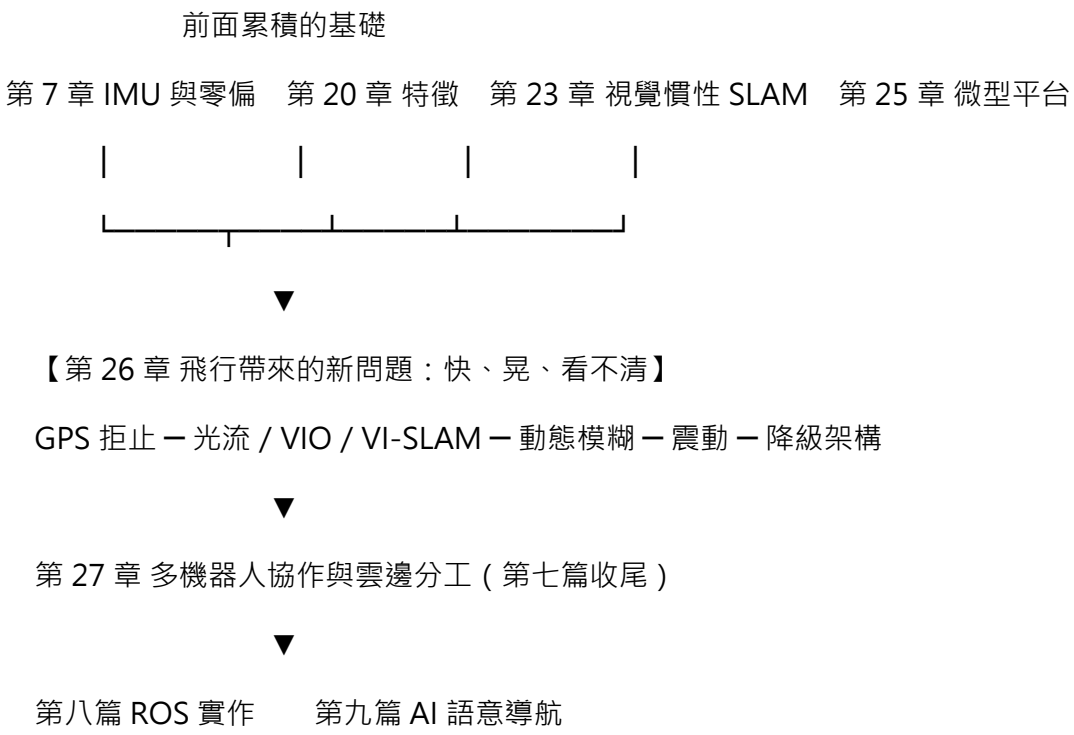
本章在全書的位置

第 25 章把 SLAM 壓進了毫瓦級的運算預算。本章換一個維度：速度。

地面機器人的典型速度是每秒 0.5 公尺，而無人機可以是它的十倍甚至五十倍。這個差異不是「同一套方法跑快一點」而已——它會讓第 20 章賴以生存的清晰角點糊成一片、讓第 22 章倚重的中期資料關聯根本來不及建立、也會讓第 7 章談過的 IMU 零偏以一種意想不到的方式被螺旋槳製造出來。

本章的三張圖就在量這些事：圖 26-1 比較光流、VIO 與 VI-SLAM 三層技術的漂移行為；圖 26-2 把 26.4 節的「三重打擊」拆開來量，並找出最有效的那一個對策；圖 26-3 則說明一件反直覺的事——提高 IMU 取樣率救不了震動造成的混疊。

圖 26-0　第 26 章的連結地圖



學習目標

上一章我們看到 SLAM 如何被壓縮進毫瓦級的預算。本章聚焦無人機導航本身的挑戰：在天空中，機器人失去了地面的支撐、擁有完整的六自由度、速度快得多、而且任何定位失誤都

可能導致墜機。學完本章之後，同學應能夠達成下列目標：

8. 說明無人機定位相對地面機器人的四項特殊困難。
9. 說明 GPS 拒止環境的成因與典型場景，並列舉可用的定位手段。
10. 比較光流、VIO 與視覺 SLAM 三種技術的能力層次與適用場合。
11. 說明高速運動、震動與影像模糊如何影響視覺定位，並提出對策。
12. 規劃一次無人機室內導航實驗的流程與安全措施。
13. 說明 EuRoC 與 TUM-VI 兩大視覺慣性資料集的特性與用途。
14. 說明從微型無人機到視覺導航無人機的技術演進脈絡。

核心概念

中文名詞	英文名詞	概念說明
微型飛行器	MAV	小型無人飛行器，是視覺慣性導航研究的主要平台。
GPS 拒止環境	GPS-denied	衛星訊號被遮蔽或干擾而無法使用的環境。
光流	Optical Flow	影像中亮度模式隨時間移動的向量場。
光流定位	Optical Flow Positioning	以向下相機的光流估計水平速度以維持懸停。
視覺慣性里程計	VIO	緊耦合相機與 IMU 估計運動，會累積漂移。
視覺慣性 SLAM	VI-SLAM	VIO 加上建圖與閉環，可消除長期漂移。
動態模糊	Motion Blur	曝光期間相機移動造成的影像模糊。
特徵軌跡縮短	Shortened Feature Tracks	高速下特徵快速離開視野，追蹤時間變短。
滾動快門	Rolling Shutter	感測器逐列曝光，高速時造成影像變形。
全域快門	Global Shutter	全畫面同時曝光，適合高速運動。

中文名詞	英文名詞	概念說明
積極機動	Aggressive Maneuver	大角度、高加速度的飛行動作。
光度校正	Photometric Calibration	校正相機的亮度響應與暗角，對直接法重要。
動作捕捉系統	Motion Capture	以外部相機提供毫米級真值位姿的實驗設備。

26.1 無人機定位的特殊困難

第 3.6 節曾比較過五種平台的導航需求，本節深入無人機為何是其中最嚴苛的一種。

四項特殊困難

- 完整的六自由度運動：地面機器人在平面上運動，只有三個自由度（第 10.3 節的 x、y、θ）；無人機能在三維空間中任意平移與旋轉。狀態變多、運動更複雜，估測難度直接上升。
- 沒有里程計：這是最容易被忽略卻極為關鍵的一點。輪型機器人有輪速編碼器提供里程計（第 3、13 章），即使不完美也是穩定的運動參考；無人機在空中沒有任何接觸，完全沒有等效的感測器。它只能靠 IMU 積分——而第 7.3 節告訴我們，IMU 單獨使用會迅速漂移。
- 速度快、動態劇烈：無人機的飛行速度遠高於室內地面機器人，而且姿態變化劇烈。這直接導致影像模糊與特徵追蹤困難（26.4 節）。
- 失誤的後果嚴重：輪型機器人定位錯誤最多是撞牆停下；無人機定位錯誤可能直接墜機，造成設備損壞甚至人員傷害。這使得系統的穩健度要求遠高於精度要求。

載重與電力的雙重限制

第 25 章已詳述受限平台的問題，對無人機而言更為尖銳：每增加一公克的感測器重量，就要消耗額外的推力與電力；而飛行本身已經消耗了絕大部分電量。這使得無人機的感測器選擇極度受限——這也是第 9.5 節指出「相機加 IMU 是無人機最理想組合」的根本原因。

即時性的嚴苛要求

地面機器人的定位若延遲 0.5 秒，它只是慢半拍地停下；無人機的定位若延遲 0.5 秒，在飛行過程中可能已經移動了數公尺，控制迴路會失去穩定。

因此無人機系統對延遲的容忍度極低。這解釋了為什麼第 25 章提到 NanoSLAM 要把端到端執行時間壓在 250 毫秒以內，也解釋了為什麼高階飛控的姿態控制迴路要以數百赫茲

執行——這正是第 7 章 IMU 高頻輸出價值的體現。

26.2 GPS-denied 環境導航

第 5 章詳細討論過 GNSS 的原理與限制，本節聚焦無人機在衛星訊號不可用時的處境與對策。

典型的 GPS 拒止場景

5. 室內：建材完全遮蔽衛星訊號（第 5.5 節）。工廠、倉庫、商場、體育館的巡檢與盤點任務都屬此類。
6. 橋樑與結構物下方：橋樑檢測是無人機的重要應用，但橋面本身就是遮蔽物。
7. 隧道與地下空間：礦坑、下水道、地下停車場——完全沒有衛星訊號。
8. 都市峽谷：第 5.5 節提過的高樓夾道，不只遮蔽，還有嚴重的多路徑造成錯誤定位——這比完全沒訊號更危險，因為系統可能不知道自己錯了。
9. 森林與植被下：樹冠遮蔽使訊號微弱且不穩定。
10. 人為干擾：第 5.5 節提過的 Jamming 與 Spoofing，對無人機是嚴肅的資安議題。

可用的定位手段

在 GPS 拒止環境中，無人機能倚賴的定位手段依可靠度排序如下：

手段	提供什麼	限制	本書章節
IMU	高頻姿態與運動	單獨使用快速漂移	第 7 章
氣壓計	相對高度	受氣流與溫度影響	本章
光流（向下相機）	水平速度	需地面紋理，不提供絕對位置	26.3 節
測距感測器（向下）	離地高度	地面起伏會誤判	第 8 章

手段	提供什麼	限制	本書章節
VIO（相機 + IMU）	完整 6-DoF 位姿	會漂移，需紋理與光線	第 23 章
VI-SLAM	位姿 + 地圖 + 閉環	運算量較大	第 23 章
UWB / 視覺標記	絕對位置校正	需事先部署基礎設施	第 4 章

表 26-1　GPS 拒止環境中無人機的定位手段

分層策略

實務系統採用分層設計，把這些手段組合起來：

11. 底層（數百 Hz）：IMU 提供姿態，維持飛行穩定。這一層不能有任何中斷，否則直接失控。
12. 中層（數十 Hz）：光流與測距提供速度與高度，支撐定點懸停。即使沒有地圖，無人機也能「待在原地不飄走」。
13. 上層（10 至 30 Hz）：VIO 或 VI-SLAM 提供完整位姿與地圖，支撐路徑規劃與自主導航。
14. 校正層（不定期）：UWB、視覺標記或回到 GPS 可用區域時，提供絕對位置校正（第 4、5 章的融合思想）。

這個分層的重要性在於容錯：若上層的 VIO 失效，無人機仍能靠中層的光流維持懸停，不至於墜機；若中層也失效，底層的 IMU 至少能維持姿態穩定，讓操作者接手。安全設計的核心原則，就是讓失效逐級降級而非瞬間崩潰。

26.3　Optical Flow、VIO、Visual SLAM

本節釐清三種視覺定位技術的能力層次——它們常被混用，但提供的資訊差異很大。

光流：最基本的一層

第 9.4 節介紹過光流是「影像中亮度模式隨時間的移動」。在無人機上，它有一個極為實用的應用：向下光流定位。

做法是在機腹裝一顆朝下的相機，持續計算地面紋理在畫面中的移動速度。配合測距感測器提供的離地高度，就能換算出無人機的水平移動速度。

15. 提供什麼：水平速度（不是位置）。
16. 能做什麼：定點懸停。若光流顯示地面在往後移，表示無人機在往前飄，飛控就往後修正。這讓無人機能在沒有 GPS 的室內穩定停在原地。
17. 限制：只給速度，積分後才是位置，因此仍會漂移；需要地面有足夠紋理（在純白地板或水面上會失效）；高度量測不準會直接造成速度換算錯誤。

光流定位是消費級無人機室內懸停的標準配備，運算量極小，是「用最少資源解決最關鍵問題」的好例子。

VIO：完整的六自由度里程計

VIO 緊耦合相機與 IMU（第 7.6、23.3 節），輸出完整的 6-DoF 位姿與速度。相較光流，它提供的資訊完整得多，能支撐路徑追蹤與自主飛行。

但 VIO 本質上仍是里程計——它會累積漂移（第 20.1 節提過視覺里程計的特性）。飛行時間一長，估測位置就會偏離真實位置。

視覺慣性 SLAM：加上地圖與閉環

VI-SLAM 在 VIO 之上加入建圖與閉環偵測（第 23 章），能在重訪同一地點時修正累積漂移。代價是運算量與記憶體需求上升。

技術	輸出	會漂移嗎	運算需求	典型用途
光流	水平速度	積分後會	極低	定點懸停

技術	輸出	會漂移嗎	運算需求	典型用途
VIO	6-DoF 位姿 + 速度	會累積	中	路徑追蹤、自主飛行
VI-SLAM	位姿 + 地圖 + 閉環修正	閉環可修正	較高	探索建圖、長時間任務

表 26-2　三種視覺定位技術的能力比較

如何選擇

18. 任務是「懸停拍攝」：光流就夠了，省電省運算。
19. 任務是「飛一段預定航線」：需要 VIO 提供位置追蹤。
20. 任務是「探索未知空間並建圖」：需要 VI-SLAM。
21. 任務是「長時間反覆巡檢同一場域」：VI-SLAM 加上地圖重用（第 23 章 Atlas 的多階段作業）最為合適。

這個層次選擇的思維，呼應了本書反覆強調的原則：不要用超過任務需求的技術，因為每一分運算與電力，在無人機上都是飛行時間的代價。

量一量：三層技術各會漂多少

表 26-2 用「會漂移嗎」這一欄區分了三種技術，但沒說會漂多少、以什麼形式漂。本小節用一次完整的飛行模擬把它量出來。

設定是無人機沿一條橢圓航線繞六圈、共 72 秒，平均速度約每秒 2.5 公尺。三種估測各依自己的物理特性建模：光流量到的是機體座標的水平速度，必須乘上測距感測器量到的高度才能換算成真實速度（高度有 4% 誤差），再用陀螺儀積分出的航向轉回世界座標（航向會漂）；VIO 直接輸出相對位姿，誤差逐步累積；VI-SLAM 在 VIO 之上，每繞完一圈就與前一圈的同一位置建立閉環，做位姿圖優化。每種設定各跑五次取中位數。

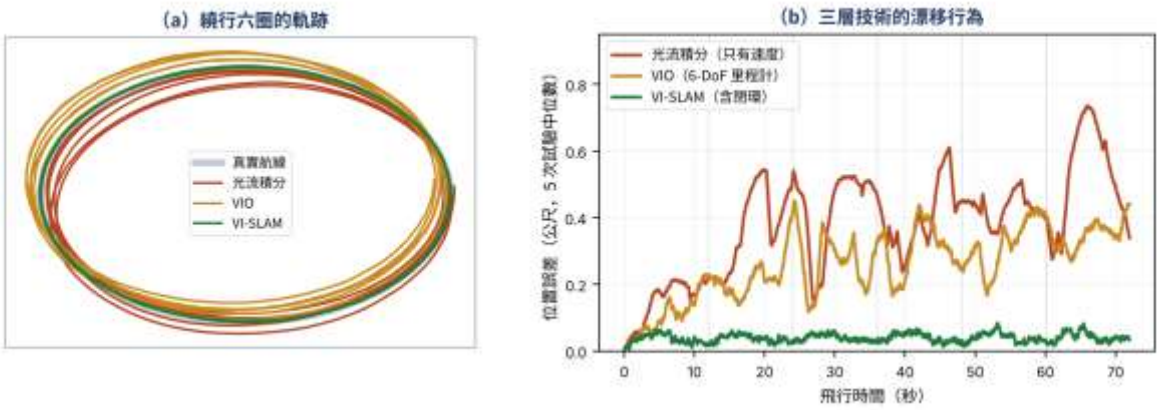


圖 26-1　光流、VIO 與 VI-SLAM 的漂移行為

（a）是六圈疊在一起的軌跡。綠色的 VI-SLAM 幾乎與真實航線重合，六圈畫在一起看不出分岔；黃色的 VIO 一圈一圈往外偏；紅色的光流則偏得更明顯，而且偏的方向不固定——這是航向誤差在轉動整個速度向量的後果。

（b）是位置誤差隨時間的變化。後半段（36 秒之後）的平均誤差是：光流 0.452 公尺、VIO 0.324 公尺、VI-SLAM 0.041 公尺；峰值分別是 0.736、0.441 與 0.084 公尺。VI-SLAM 比前兩者好了 8 到 11 倍。

但更值得注意的是三條線的「形狀」而不是高度。綠線從頭到尾是平的——它的誤差不隨飛行時間成長，而是被閉環鎖在一個固定的水準上。紅線與黃線則呈現明顯的鋸齒：每繞一圈誤差先上升、再部分回落。回落不是因為被修正了，而是因為航線是封閉的，繞回起點附近時累積的誤差碰巧部分抵消——這是巧合，不是機制。若航線是單程的，這兩條線只會一路往上。

這張圖也說明了 26.3 節「如何選擇」那幾條建議的根據。懸停時不需要位置、只需要速度，光流的漂移完全不構成問題；但只要任務需要「飛到某個座標」或「飛完就回來」，沒有閉環的方案就會在幾十秒內累積出公尺級的誤差。

> **光流為什麼會比 VIO 還糟**
>
> 表面上看，光流的量測比較單純，誤差應該比較小。但它有兩個 VIO 沒有的弱點。

第一，它只給速度，沒有任何姿態資訊，所以要把速度轉到世界座標必須借用陀螺儀積分出的航向——而航向誤差會直接旋轉整個速度向量，造成的位置誤差與飛行距離成正比，而不是與時間的平方根成正比。

第二，它的速度換算依賴高度量測。本模擬只給了 4% 的高度誤差，就造成全程 4% 的尺度偏差。若無人機飛過一張桌子，測距感測器量到的高度突然變小，光流會誤判成「飛得更慢」——這是消費級無人機室內飄移最常見的原因。

26.4 高速運動、震動與影像模糊

本節是無人機視覺定位最核心的技術挑戰。

三重打擊

研究指出，現有的 VIO 系統在高速下會因為動態模糊、特徵軌跡縮短與控制迴路延遲增加而效能劣化。這三者恰好構成一個惡性循環：

22. 動態模糊：曝光期間相機移動，影像變糊。在如此的速度下，新的問題會浮現，既有的困難也被加劇——動態模糊變得更為明顯，損害視覺特徵追蹤。模糊的影像中，角點變成一片糊，特徵提取直接失敗（第 20 章的特徵需要清晰的角點）。
23. 特徵軌跡縮短：高速飛行時，同一個特徵點快速掃過視野就消失了。第 23.2 節強調的「中期資料關聯」因此難以建立，系統只能倚賴短期關聯，精度下降。
24. 控制迴路延遲：視覺處理本身需要時間，高速下這個延遲對應的位移更大，控制品質惡化，可能引發震盪。

資料集反映的現實

這些挑戰在標準評測資料集上有清楚的呈現。有研究觀察到，由於無人機的高速移動，影像會有些模糊，在困難序列如 MH_04、MH_05、V1_03 與 V2_03 中尤其明顯。

更有趣的是一個細節：EuRoC 的機房序列在開頭都有快速運動以協助 IMU 初始化（第

23.4 節），但由此產生的動態模糊對視覺追蹤是一大挑戰；因此 DSO 的作者裁掉開頭，只使用無人機在空中的部分，但對 VIO 系統而言 IMU 初始化不可或缺，無法捨棄開頭。

這個細節同時說明了兩件事：第一，IMU 初始化需要運動激勵（第 23.4 節反覆強調），但運動激勵本身會造成模糊，兩者互相拉扯；第二，不同方法在同一資料集上的評測條件可能不同，同學閱讀論文比較數據時要留意這類細節。

速度的量級

讓同學對「高速」有具體概念：EuRoC 微型飛行器資料集的速度很少超過每秒 5 公尺；競速無人機資料集可達每秒 28 公尺，但通常是室內結構化場地或短程戶外賽道的 RGB 相機。有研究以雙目 RGB 的 VIO 支援到每秒 17.5 公尺，但它依賴雙目視覺並假設有利的光線條件。

對照之下，室內地面機器人的速度通常在每秒 0.5 公尺以下——無人機的速度是它的十倍甚至數十倍。這就是為什麼無人機的視覺定位困難得多。

五種對策

25. 提高快門速度：縮短曝光時間直接減少模糊，代價是影像變暗，需要更好的感光元件或更亮的環境。
26. 使用全域快門：滾動快門逐列曝光，高速旋轉時會造成影像歪斜；全域快門全畫面同時曝光，是高速應用的必要選擇。
27. 倚重 IMU：有研究整合以 IMU 為基礎的運動預測模組來改善跨幀的特徵追蹤，特別是在高速運動期間。IMU 能預測特徵「應該移動到哪裡」，大幅縮小搜尋範圍，提升追蹤成功率。這是第 7.6 節「IMU 為視覺提供初值」的具體實現。
28. 影像增強：對模糊或低對比的影像做預處理。有研究結合對比度受限的自適應直方圖等化（CLAHE）、梯度濾波與描述子驗證，以在動態模糊下維持穩定的特徵軌跡。

29. 自適應權重：依影像品質動態調整視覺與慣性的信任比例。有研究在 EuRoC 與 TUM VI 資料集以及實地測試中驗證了自適應共變異數方法，在有動態模糊的情境中顯著改善無人機的導航精度。

震動的問題

除了飛行速度，螺旋槳與馬達造成的高頻震動也是無人機特有的問題：

30. 對 IMU：震動會污染加速度計讀值，若頻率接近取樣率還可能造成混疊。實務上需要機械減震（軟膠墊）與濾波處理。
31. 對相機：高頻震動造成的微小模糊會降低特徵品質，通常以減震座與較快的快門緩解。
32. 對安裝：感測器與機身的相對位置（外參，第 10 章）若因震動而鬆動改變，整個系統的精度都會受影響。

量一量：三重打擊各打掉多少

26.4 節把高速飛行的困難歸納為動態模糊、特徵軌跡縮短與控制迴路延遲三者。這一節把前兩者分開來量，順便檢驗五種對策裡哪一個最有效。

實驗方法：合成一張帶有多尺度紋理與幾何邊角的地面影像（模擬磚縫、標線、門框），依「速度 × 曝光時間 × 焦距 ÷ 深度」算出對應的像素位移，施以線性運動模糊，再用 ORB 偵測與交叉比對，統計有多少特徵能與清晰影像正確配對。相機參數取無人機常用的廣角配置（焦距 380 像素），場景深度 4 公尺。

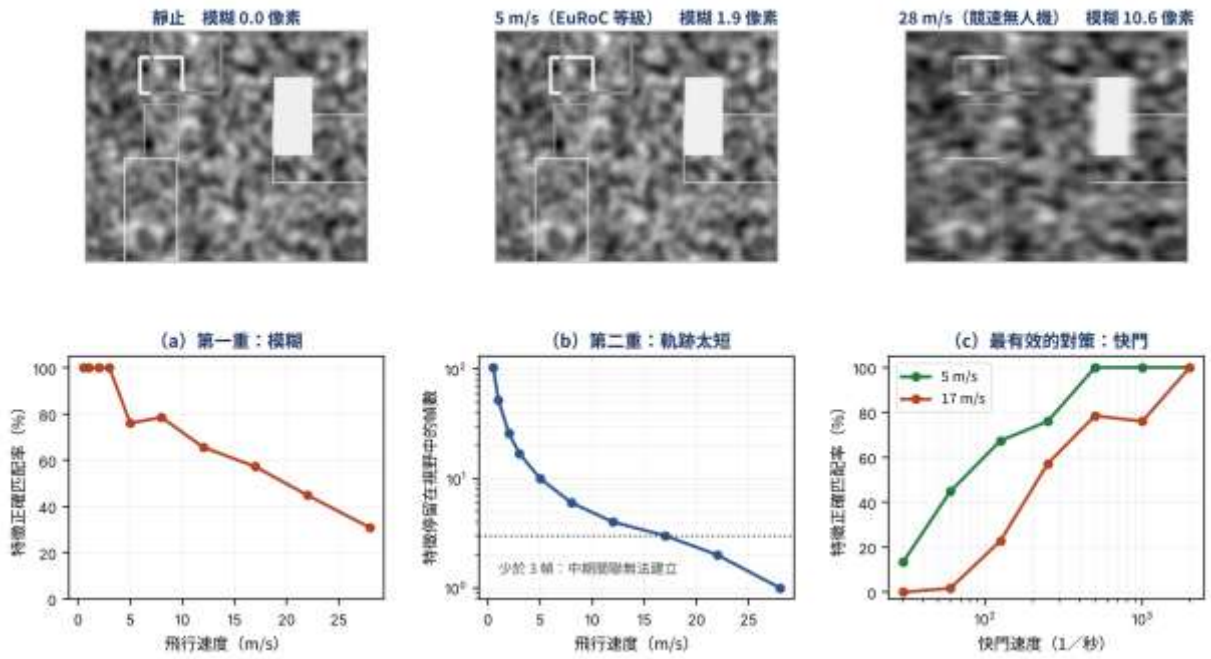


圖 26-2　速度造成的三重打擊，以及最有效的對策

上排是同一塊紋理在三種速度下的樣子。靜止時角點清晰；每秒 5 公尺（EuRoC 微型飛行器資料集的典型速度）時模糊 1.9 像素，肉眼已經看得出邊緣變軟；每秒 28 公尺（競速無人機）時模糊 10.6 像素，細紋理幾乎完全消失，只剩大塊的明暗。

（a）量化第一重打擊。每秒 3 公尺以內特徵匹配率還是 100%，5 公尺時降到 76%，12 公尺 66%，17 公尺 57%，28 公尺只剩 31%。請注意轉折點落在每秒 3 到 5 公尺之間——這恰好就是一般室內無人機的巡航速度，也解釋了為什麼 EuRoC 的困難序列正是那些飛得比較快的。

（b）量化第二重打擊，而這一重其實更致命。縱軸是一個特徵點從進入視野到離開視野能被看到幾幀（相機 30 fps）。每秒 0.5 公尺時是 104 幀（3.5 秒），2 公尺時 26 幀，5 公尺時 10 幀，17 公尺時只剩 3 幀，28 公尺時——只剩 1 幀。

一幀是什麼意思？意思是這個特徵在這一幀被偵測到，下一幀它已經不在畫面上了。沒有第二次觀測，就沒有三角測量（第 21.5 節需要視差）、沒有中期資料關聯（第 23.2 節）、也沒有辦法建立地圖點。換句話說，在每秒 28 公尺的速度下，就算影像完全清晰，視覺 SLAM 一樣做不下去。模糊只是讓情況雪上加霜。

（c）檢驗 26.4 節提到的第一種對策：提高快門速度。以每秒 17 公尺為例，快門 1/30 秒時匹配率是 0%，1/60 秒 1.7%，1/125 秒 22.9%，1/250 秒 57.2%，1/500 秒 78.5%，1/1000 秒 76.0%，1/2000 秒 100%。從 1/125 提高到 1/500，匹配率從兩成多跳到近八成——這是五種對策裡成本最低、效果最立即的一個。

代價當然也在：快門每快一倍，進光量就少一半。所以無人機的高速視覺定位實際上是一個光學問題——要嘛環境夠亮，要嘛換大光圈鏡頭與高感光元件。這也是為什麼競速無人機的視覺研究多半限定在「室內結構化場地或光線良好的戶外賽道」。

> **一個對專題的務實建議**
>
> 把（a）與（b）放在一起看，會得到一個很清楚的速度上限：每秒 3 公尺以內，模糊與軌跡長度都還健康（100% 匹配、17 幀以上）；每秒 5 公尺是堪用的邊界（76%、10 幀）；超過每秒 10 公尺，就必須同時處理快門、光源與演算法三件事。
>
> 同學在規劃無人機專題時，與其追求飛得快，不如先把速度限制在每秒 2 到 3 公尺，把定位與建圖做穩；速度是最後才該放寬的那個參數。

量一量：螺旋槳的震動會變成什麼

26.4 節最後提到震動「會污染加速度計讀值，若頻率接近取樣率還可能造成混疊」。這句話值得展開，因為混疊的後果比「雜訊變大」嚴重得多。

實驗設定：在一段緩慢的真實加速度訊號上疊加螺旋槳震動（基頻加二次諧波，振幅 2.5 公尺 / 秒²），以 400 Hz 或 1000 Hz 取樣，比較取樣後的訊號與真值的平均差——這個平均差，在下游的估測器眼中就是一個加速度計零偏。

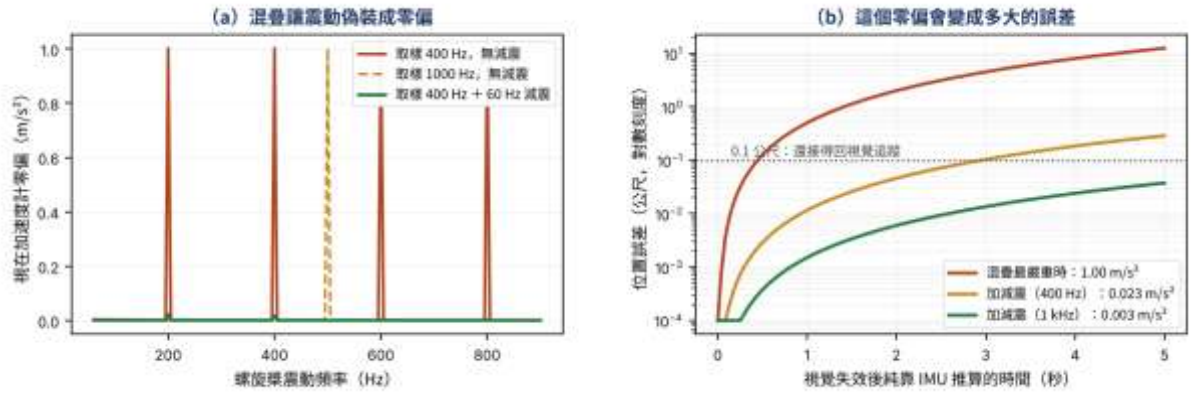


圖 26-3　震動的混疊，以及它為什麼偽裝成零偏

（a）掃描震動頻率。紅線是 400 Hz 取樣、沒有減震：大多數頻率下視在零偏接近零，但在 200、400、600、800 Hz 這幾個點上突然竄到 1.00 公尺 / 秒²。這些正是取樣率的整數倍與半整數倍——震動在這些頻率上被折疊成直流分量，也就是一個看起來完全像零偏的常數偏移。

黃色虛線是把取樣率提高到 1000 Hz。尖峰的位置改變了，但高度完全沒變——同樣是 1.00 公尺 / 秒²。這是本圖最重要的訊息：提高取樣率不能解決混疊，只能把問題搬到別的頻率去。這是一個相當常見的誤解。

綠線是加上 60 Hz 截止頻率的機械減震（軟膠墊加類比低通）。視在零偏的最大值從 1.00 降到 0.023 公尺 / 秒²，降了四十倍以上。關鍵在於這個濾波發生在「取樣之前」——高頻成分在進入類比數位轉換器之前就被擋掉了，因此根本沒有機會被折疊。取樣之後再做數位濾波，對混疊完全無效，因為訊號已經壞了。

（b）說明這 1.00 公尺 / 秒² 有多嚴重。零偏積分兩次就是位置誤差：視覺失效 1 秒會產生 0.50 公尺的誤差、2 秒 2.0 公尺、4 秒 8.0 公尺。對照第 23 章圖 23-1 的結論——零偏估到 ±0.01 時 2 秒的誤差只有 0.057 公尺——震動造成的這個「假零偏」比那裡討論的最差情況還糟三倍以上。

更麻煩的是，這個偏移不是隨機的，而是隨轉速改變的。無人機加速時轉速上升、震動頻率跟著移動，假零偏可能從零突然跳到 1.00 再掉回零。第 23.4 節的 IMU 零偏估計假設零偏

是緩慢變化的，面對這種跳變完全無能為力。所以減震不是「調校的細節」，而是系統能不能運作的前提。

一個可以在實驗室做的檢查

把無人機固定在桌上（槳葉拆掉或加裝防護），從怠速緩慢加到全速，同時記錄加速度計原始讀值。

然後對每個轉速區間算平均值。如果平均值在某幾個轉速上明顯偏離重力，那就是混疊——而且你剛好找到了必須避開的轉速區間。

更完整的做法是對原始訊號做快速傅立葉轉換，看看螺旋槳的基頻與諧波落在哪裡，再據此選擇減震座的軟硬度（也就是它的截止頻率）。這個實驗只需要一台無人機與一支記錄程式，卻能省下後續數週的除錯時間。

26.5　無人機室內導航實驗

本節說明如何規劃一次安全而有教學價值的無人機室內導航實驗，這也是專題實作的基礎。

安全第一：必須先講的事

無人機實驗與地面機器人最大的差異在於風險。以下是不可妥協的原則：

33. 防護網或防護籠：室內實驗應在有防護網的空間進行，或使用附有槳葉保護框的機型。
34. 選用微型平台：第 25 章介紹的微型無人機重量僅數十公克，即使失控撞到人也幾乎不會受傷。教學實驗強烈建議從微型平台開始。
35. 緊急停止：操作者必須隨時能一鍵切斷動力，且此功能不依賴需要除錯的自主系統。
36. 分階段測試：先在模擬中驗證、再繫繩測試、最後才自由飛行。跳過階段是意外的主因。

37. 人員位置：實驗時所有非必要人員應在防護網外，操作者站在無人機後方而非前方。

實驗場地的準備

38. 紋理：地面與牆面需有足夠的視覺紋理（第 9、20 章的白牆詛咒在無人機上同樣成立）。必要時可在地面鋪設有圖案的墊子。

39. 光線：均勻且充足，避免逆光與強烈明暗對比。窗戶可能造成過曝，建議拉上窗簾以維持穩定的照明。

40. 氣流：避免冷氣直吹或開窗形成的氣流，它會嚴重干擾微型無人機的穩定。

41. 真值系統（選用）：若有動作捕捉系統，能提供毫米級的真值位姿，用於量化評測（26.6 節）。沒有的話，可用第 4 章的 AprilTag 提供近似真值（第 24.3 節的第四種用途）。

實驗流程

4. 系統檢查：確認 IMU 校正、相機校正（第 9.2 節）、感測器外參（第 10 章）、以及通訊連線正常。

5. 懸停測試：最基本的能力。啟動光流定位，讓無人機在原地懸停 30 秒，量測位置漂移。若懸停都不穩，後續的自主飛行不必嘗試。

6. VIO 初始化：依第 23.4 節，以帶加減速與旋轉的運動完成 IMU 初始化，確認尺度估計正確（可用已知距離驗證）。

7. 簡單航線：設定一條簡單的方形或直線航線，讓無人機自主飛行，記錄軌跡。

8. 閉環測試：讓無人機繞一圈回到起點，量測終點與起點的落差，評估累積漂移（第 13.7 節的評估方法）。

9. 挑戰測試：逐步增加難度——提高速度、加入急轉、暫時遮蔽相機——觀察系統在什麼條件下失效。這是最有教學價值的環節。

資料記錄

每次飛行都應完整記錄：影像、IMU 原始資料、飛控狀態、估測軌跡、以及（若有）真值軌跡。這些資料能離線反覆分析，也能作為自製資料集——對專題研究而言，一份自己場域的資料集本身就是有價值的成果。

26.6 視覺慣性資料集：EuRoC、TUM-VI

沒有無人機硬體，同樣能做無人機 SLAM 的研究——這正是公開資料集的價值。本節介紹兩大標準資料集。

EuRoC MAV 資料集

第 7 章案例 7-3 已介紹過 EuRoC，此處補充細節。它包含十一段以微型飛行器錄製的序列，提供雙目相機影像、IMU 量測與來自動作捕捉系統的真值位姿。

每段序列涵蓋 30 至 60 秒的飛行，包含來自外部動作捕捉的高保真 6-DoF 真值位姿，以及運作環境的高解析度點雲掃描，使軌跡估計與稠密三維地圖重建都能以幾何一致的參考資料進行評估。典型的評測流程是以 20 Hz 的單目影像與 200 Hz 的 IMU 資料，交由特徵法 VIO（例如關閉閉環的 ORB-SLAM3）處理，產出具真實尺度的相機軌跡與稀疏三維地標。

42. 序列分類：機房（MH）序列在工業廠房錄製，房間（V1、V2）序列在有動作捕捉的實驗室錄製。

43. 難度分級：每組序列都分 easy、medium、difficult 三級，難度來自速度、光線與運動劇烈程度。這讓同學能循序漸進地測試演算法。

44. 使用建議：從 MH_01_easy 開始建立成功經驗，再挑戰 V1_03_difficult 等困難序列。

TUM-VI 資料集

TUM-VI 是另一套廣泛使用的基準。它提供 1024×1024 解析度、20 Hz 的相機影像，具備高動態範圍與光度校正；IMU 以 200 Hz 量測三軸加速度與角速度，相機與 IMU 在硬體層級時間同步。

相較之下，EuRoC MAV 的序列較短、變化較少，只包含一個機房與一個實驗室的錄製；此外 EuRoC MAV 不含光度校正，而這對評測直接法（第 20.2 節）相當重要。

45. 特色：涵蓋室內外多種場景、序列較長、有光度校正、時間同步在硬體層級完成。

46. AR / VR 相關：第 23.7 節引述過 ORB-SLAM3 在 TUM-VI 房間序列的毫米級精度，那正是這個資料集的房間場景。

特性	EuRoC MAV	TUM-VI
平台	微型飛行器	手持感測器組
影像	雙目，20 Hz	雙目 1024×1024,20 Hz
IMU	200 Hz	200 Hz，硬體時間同步
真值	動作捕捉 / 雷射追蹤	動作捕捉（部分序列）
光度校正	無	有
場景多樣性	一個機房 + 一個實驗室	室內外多種場景
額外資料	環境點雲掃描	高動態範圍影像

表 26-3　EuRoC 與 TUM-VI 資料集比較

資料集的使用建議

47. 演算法比較：在同一資料集上比較不同系統（第 23 章實作 23-1 的做法），數據才有可比性。

48. 留意評測條件：如 26.4 節提過的「是否裁掉開頭」，不同論文的評測條件可能不同，直接比較數字要謹慎。

49. 自製資料集：公開資料集無法涵蓋你的特定場域。用第 26.5 節的方法錄製大學場域的資料，是專題極有價值的貢獻。

50. 新興資料集：近年也出現速度更快、感測模態更多的資料集，如競速無人機資料集（速度可達每秒 28 公尺）與含熱像的資料集，適合進階研究。

26.7 從 nanoSLAM 到視覺導航無人機

本節把第 24、25 兩章連起來，梳理無人機自主導航的技術演進與未來方向。

三個層次的無人機自主

層次	能力	關鍵技術	代表平台
穩定	不飄走、能懸停	IMU + 光流 + 測距	消費級無人機
自主	能沿航線飛、能避障	VIO + 路徑規劃	專業無人機
智慧	能探索、建圖、理解任務	VI-SLAM + 語意 + VLM	研究平台、未來產品

表 26-4 無人機自主能力的三個層次

這個層次表能幫同學定位自己的專題目標。多數教學專題應從「穩定」層次確認基礎，再挑戰「自主」層次；「智慧」層次適合研究所或長期專題。

技術演進的三條主線

10. 運算的下沉：從外部電腦運算（卸載）、到機載高性能運算（如 Jetson）、再到第 25 章的毫瓦級專用晶片。這條線讓自主能力得以進入更小、更便宜的平台。
11. 感測的輕量化：從笨重的光達、到相機加 IMU、再到第 25 章的 ToF 矩陣感測器。這條線降低了重量與功耗的門檻。
12. 智慧的上升：從幾何導航、到語意理解（第 24 章）、再到 VLM 驅動的任務級自主。這條線提升了機器人能執行的任務複雜度。

值得注意的是，這三條線正在交會：第 25 章的微型平台已經能執行 YOLO 物件偵測（低於 100 毫瓦、最短 17 毫秒），意味著「小、省電、且有語意理解能力」的無人機正在

成為現實。

大學場域的專題方向

51. 室內測繪：以微型無人機掃描系館內部（如挑高中庭、樓梯間），產生三維模型。這些是地面機器人難以到達的空間，能展現無人機的獨特價值。
52. 設施巡檢：屋頂太陽能板、外牆、高處管線的檢查——無人機能安全地取代人員登高作業。
53. 農業應用：呼應第 25 章案例 25-3，台灣的果園與溫室是微型無人機的理想場域。
54. 教學資料集：錄製並公開一份大學場域的視覺慣性資料集，供後續學生與研究者使用。這是相對容易入手、卻能長期產生價值的貢獻。
55. 地面與空中協作：讓機器狗（第 18、24 章）與無人機協同探索——地面機器人續航長、承載大，無人機視野廣、能到高處。這是第 27 章多機器人協作的具體應用。

前沿方向

最後提幾個值得關注的研究方向，供有志深入的同學參考：

56. 高速飛行的感知：如 26.4 節所述，現有 VIO 在高速下會劣化。以熱像儀等非傳統感測器突破速度限制，是活躍的研究方向。
57. 學習與幾何的結合：第 20.7 節提過深度學習特徵正進入 SLAM。有研究整合深度特徵提取器與稀疏特徵匹配器，並結合傳統的手工關鍵點偵測與光流追蹤，以 TensorRT 加速推論，在快速相機運動與光線變化下改善前端追蹤效能。
58. 學習式慣性里程計：純 IMU 的學習方法在特定情境下已能接近 VIO 的表現——這對視覺失效的場景（黑暗、煙霧）是重要的備援。
59. 群體協作：多台微型無人機協同建圖與探索，是第 27 章的主題，也是本篇的最後一塊拼圖。

至此，無人機導航的核心挑戰與技術路線已完整呈現。下一章我們把視野從單機擴大到

群體：多台機器人如何協同建圖？各自的地圖如何融合？雲端與邊緣如何分工？通訊、同步與安全又有哪些課題？第 27 章將完成第七篇的最後一塊拼圖。

26.8　無人機視覺定位的現況：速度的上限在哪裡

本章量了很多「會壞掉」的地方。作為收尾，本節整理目前的技術邊界，以及突破它的三條路。

一、現況的三個速度帶

速度帶	典型場景	視覺定位的狀態	主要限制
每秒 3 公尺以內	室內巡檢、拍攝	成熟，VI-SLAM 可穩定運行	運算與續航
每秒 3 至 10 公尺	戶外巡檢、測繪	可行，需快門與減震的配套	動態模糊
每秒 10 至 20 公尺	競速、快速搜救	研究階段，需雙目與良好光線	特徵軌跡過短
每秒 20 公尺以上	競速極限	視覺幾乎無法單獨支撐	一幀即消失

表 26-5　無人機視覺定位的四個速度帶

這張表的分界點，正是圖 26-2 量出來的那兩個轉折：每秒 3 到 5 公尺之間模糊開始咬人，每秒 17 公尺左右特徵軌跡跌破三幀。兩個限制的性質不同——前者可以用光學手段緩解，後者是幾何上的硬限制。

二、三條突破的路

第一條是換感測器。事件相機不輸出影格，而是在每個像素亮度變化時輸出一個事件，時間解析度可達微秒級，原理上不存在動態模糊。它的代價是輸出形式完全不同，第 20 到 23 章的整套方法都要重寫。

第二條是讓 IMU 承擔更多。26.4 節提過「以 IMU 為基礎的運動預測改善跨幀特徵追蹤」，本質上是用慣性資訊預測特徵「應該移動到哪裡」，把搜尋範圍從整張影像縮到一個小窗口。在高速下這個做法的價值特別大——但它的前提是 IMU 的零偏要估得準，而圖 26-3 剛好說明了震動會怎麼破壞這件事。兩者互相牽制，這是無人機系統設計最棘手的一環。

第三條是接受限制、改變任務。既然高速時視覺撐不住，那就在高速段只靠 IMU 與氣壓計維持姿態，定期減速到每秒 2 到 3 公尺做一次視覺定位修正。這與第 19.6 節巡檢機器人的地圖維護、第 25.8 節的事件驅動運算是同一個思路——不是讓系統時時刻刻都達到最高標準，而是讓它在需要的時候達到就好。

對同學的專題而言，第三條路最務實。它不需要特殊硬體，也不需要改寫演算法，只需要在任務規劃時把「什麼時候該慢下來」寫進去。

公式或流程

一、光流速度換算

向下相機量到的光流是「影像上每秒移動幾個像素」，要換算成真實的水平速度，需要離地高度與相機焦距：

$$v = (\mathit{flow_pixels} / f) \times h$$

其中 flow_pixels 是每秒的像素位移、f 是焦距（像素單位，第 9.1 節的內參）、h 是離地高度（由測距感測器提供）。

這條式子透露一個關鍵風險：高度 h 出現在分子。若測距感測器誤判高度（例如飛過桌面時高度突然變小），換算出的速度就會錯誤，飛控可能因此做出錯誤修正而發生漂移甚至撞擊。這是光流定位最常見的失效原因，同學在實驗時務必注意場地的地面起伏。

二、動態模糊的量化

曝光期間影像上的模糊長度，約為：

$$blur_pixels \approx (v / h) \times f \times t_exposure$$

其中 v 為飛行速度、h 為到場景的距離、t_exposure 為曝光時間。

代入實際數字感受一下：飛行速度 5 m/s、距離場景 5 公尺、焦距 400 像素、曝光 10 毫秒，則模糊約 4 像素——已足以嚴重影響角點偵測（第 20 章的特徵需要清晰的局部結構）。若速度提高到 20 m/s，模糊達 16 像素，特徵提取幾乎完全失效。

這條式子也直接指出對策：縮短曝光時間 t_exposure 能線性減少模糊——這正是 26.4 節第一項對策的數學依據。

三、VIO 的狀態與誤差項

無人機 VIO 的狀態向量與第 23.3 節相同，每個關鍵幀為：

$$S = \{R, p, v, b_g, b_a\} \quad (\text{共 } 15 \text{ 自由度})$$

緊耦合最佳化同時最小化視覺與慣性誤差：

$$\min \Sigma \rho(\| e_視覺 \|^2) + \Sigma \| e_慣性 \|^2$$

對無人機而言，一個實務重點是兩者權重的動態調整：當偵測到影像模糊或特徵數不足時，應降低視覺項的權重、提高慣性項的比重——這正是 26.4 節第五項對策（自適應權重）的實現方式，也呼應第 11 章卡爾曼增益「按可信度加權」的核心思想。

四、無人機 GPS 拒止導航的分層架構

圖 26-4 無人機導航的分層架構與失效降級

【校正層】不定期：UWB / 視覺標記 / 重回 GPS 區 → 絕對位置校正

↓ 失效則降級

【上層】10–30 Hz:VI-SLAM / VIO → 完整 6-DoF 位姿 + 地圖

支撐：路徑規劃、自主導航、探索建圖

↓ 失效則降級

【中層】數十 Hz：光流 + 測距 → 水平速度 + 離地高度

支撐：定點懸停（至少不飄走）

↓ 失效則降級

【底層】數百 Hz:IMU → 姿態

支撐：維持飛行穩定，讓操作者接手

【設計原則】讓失效逐級降級，而非瞬間崩潰

案例

案例 26-1 沒有輪子的代價：無人機為何比地面機器人難

同學若同時做過地面機器人與無人機的 SLAM，會強烈感受到後者困難得多。原因不只是「飛比較快」，而是一個常被忽略的結構性差異：無人機沒有里程計。

回顧本書的脈絡：第 13.4 節說明 2D SLAM 靠「里程計預測、光達校正」的循環運作；第 14.5 節指出 Gmapping 的三個前置條件中，里程計是必要的；第 18.3 節的 LOAM 也需要運動估計作為配準初值。里程計雖然會漂移，但它提供了穩定、連續、免費的運動參考。

無人機完全沒有這個奢侈品。它唯一的內部運動來源是 IMU，而第 7.3 節已經證明：消

費級 IMU 單獨積分，數秒內就會漂移數公尺。這意味著無人機的視覺系統必須「獨力承擔」原本由里程計分擔的責任——一旦視覺失效，系統就沒有可靠的後備。

這解釋了三件事：為什麼無人機領域如此重視 VIO 的緊耦合（第 23 章）、為什麼光流定位是消費級無人機的標準配備（26.3 節）、以及為什麼 26.2 節的分層降級架構如此關鍵。

對同學的啟示是：理解一個平台的「缺什麼」，往往比理解它「有什麼」更能解釋系統設計的取捨。這種從限制反推設計的思維，是工程判斷力的核心。

案例 26-2　初始化的兩難：動得不夠不行，動太多也不行

這是一個極具教學價值的矛盾，而且同學在實作時必定會遇到。

一方面，第 23.4 節反覆強調：IMU 初始化需要足夠的運動激勵，否則尺度與零偏不可觀測。靜止或等速直線移動，系統就是初始化不了。

另一方面，26.4 節提到的實例顯示：EuRoC 的機房序列在開頭都有快速運動以協助 IMU 初始化，但由此產生的動態模糊對視覺追蹤是一大挑戰。快速運動雖然滿足了慣性系統的需求，卻破壞了視覺系統的輸入品質。

這個矛盾的存在，恰恰說明了視覺慣性融合的本質：兩種感測器的需求並不一致，甚至互相拉扯。IMU 喜歡有加速度的運動，相機喜歡平穩清晰的畫面。

實務上的解法是折衷：做「中等幅度、帶加減速與旋轉、但不過快」的運動。具體而言，拿著裝置或操控無人機做小幅度的前後左右移動並轉動數秒，速度以「畫面不明顯模糊」為上限。

這個案例也提醒同學：當論文中的方法在你手上跑不起來時，問題未必在程式，而可能在「你的資料不符合方法的隱含假設」。理解方法對輸入的要求，是成功實作的前提。

案例 26-3　速度的邊界：VIO 能飛多快

「無人機視覺定位能支援多快的飛行？」這個問題的答案，近年正被不斷推進，也清楚

呈現了技術的演進。

先看基準：EuRoC 微型飛行器資料集的速度很少超過每秒 5 公尺——這是多數 VIO 論文的評測條件。以此為基準，同學看到的多數精度數據，都是在相對溫和的速度下取得的。

往上推進：有研究以雙目 RGB 的 VIO 支援到每秒 17.5 公尺，但它依賴雙目視覺並假設有利的光線條件。競速無人機資料集則可達每秒 28 公尺，但通常是室內結構化場地或短程戶外賽道的 RGB 相機。

突破限制：更前沿的研究開始改用非傳統感測器。有研究以單目熱像慣性里程計，在低空飛行中達成每秒 30 公尺的即時運作。熱像的優勢在於不受光線影響、且對某些模糊來源較不敏感。

但這條路也有代價。研究指出，雖然以深度學習為基礎的方法在熱像上展現了不錯的效能提升，但它們的推論時間仍與無人機機載硬體嚴格的延遲與功耗限制不相容——某些方法在筆電等級的 GPU 上每幀就需要超過 30 毫秒。因此手工設計的管線仍是嵌入式高速閉環飛行最實際的解法。

這個案例濃縮了本書一再出現的主題：前沿技術的精彩數據，往往建立在特定條件之上；而真正能部署的系統，必須同時滿足精度、速度、功耗與可靠度的多重約束。學會閱讀論文時同時看「達成了什麼」與「在什麼條件下達成」，是研究能力的重要一環。

實作活動

實作 26-1　無人機視覺慣性導航實驗

目的：透過公開資料集與（可選的）實體飛行，理解無人機視覺定位的挑戰與對策。第一、二部分不需硬體，適合全體；第三部分需要微型無人機平台。

第一部分：資料集分析（全體，不需硬體）

13. 下載 EuRoC 資料集，選取 MH_01_easy 與 V1_03_difficult 兩段序列，分別以第 23 章的 ORB-SLAM3 雙目慣性模式執行，用 evo 計算 ATE。
14. 比較兩者的精度差異，並播放影像觀察 difficult 序列在哪些片段出現明顯的動態模糊，對照 26.4 節的討論。
15. 模糊量化：寫程式計算每一幀影像的清晰度（可用拉普拉斯運算子的變異數），畫出「清晰度隨時間變化」的曲線，標出最模糊的片段。
16. 關聯分析：把清晰度曲線與軌跡誤差曲線畫在一起，觀察兩者是否相關——影像最模糊的時刻，是否也是誤差最大的時刻？
17. 初始化觀察：留意序列開頭的快速運動段，記錄系統完成 IMU 初始化所需的時間，並觀察此段的影像模糊程度，親身驗證案例 26-2 的兩難。

第二部分：光流與模糊模擬（全體）

18. 用 OpenCV 的 Lucas-Kanade 光流，對一段向下拍攝的影片（可用手機在桌面上方平移拍攝）計算光流，依公式一換算水平速度。
19. 高度敏感度實驗：故意把高度值設錯（例如實際 1 公尺卻填 0.5 公尺），觀察換算速度的誤差，理解 26.5 節提到的失效風險。
20. 模糊模擬：對清晰影像人為加入動態模糊（用 OpenCV 的運動模糊核），模糊長度依公式二計算，測試 ORB 特徵偵測數量隨模糊程度如何下降。
21. 繪製「模糊像素數 vs 可偵測特徵數」的曲線，找出特徵數急遽下降的臨界點。這個臨界值就是你的系統能容忍的速度上限依據。

第三部分：實體飛行（有微型無人機平台的班級）

務必先閱讀 26.5 節的安全原則，並在防護網內、以微型平台進行。

1. 懸停測試：啟動光流定位，讓無人機在原地懸停 30 秒，以地面標記量測水平漂移範圍。

2. 地面紋理實驗：分別在有圖案的地毯、純色地板、與反光地面上懸停，比較漂移程度，驗證光流對紋理的依賴。
3. 高度實驗：在不同離地高度（0.5、1.0、1.5 公尺）懸停，觀察穩定度差異，思考公式一中高度誤差的影響。
4. 簡單航線：設定一條方形航線自主飛行，回到起點後量測落差，評估累積漂移。
5. 挑戰測試：在安全前提下逐步提高飛行速度，記錄系統開始不穩定的速度，對照案例 26-3 的討論。

討論問題

6. easy 與 difficult 序列的 ATE 差多少倍？影像清晰度與軌跡誤差的相關性如何？
7. 依公式二，若你的相機曝光 5 毫秒、焦距 400 像素、距離場景 3 公尺，飛行速度多快時模糊會達到 5 像素？這對速度上限有什麼意涵？
8. 光流換算實驗中，高度誤差 50% 造成速度誤差多少？若無人機依此錯誤速度做修正，會發生什麼？
9. （有飛行者）在哪種地面上懸停最穩定？最不穩定的原因是什麼？這印證了 26.3 節的哪一項限制？
10. 依圖 26-1 的分層架構，若你的無人機在飛行中 VIO 突然失效，系統應如何反應？請設計一套降級流程。

延伸挑戰（大學部）

其一：自適應權重實作——在 VIO 系統中加入影像品質評估（如第二部分的清晰度指標），當影像模糊時自動降低視覺項權重、提高慣性項比重，在 EuRoC difficult 序列上測試是否改善精度。這對應 26.4 節的第五項對策，也是一個完整的小型研究題目。其二：大學場域資料集——依 26.5 節的方法，以微型無人機或手持感測器組錄製系館內部的視覺慣性資料（含影像、IMU、以及用 AprilTag 提供的近似真值），整理成標準格式並公開，供後續學生使用。這是一份能長期產生價值的貢獻，也是專題成果的良好呈現形式。

延伸閱讀

8. M. Burri et al., "The EuRoC Micro Aerial Vehicle Datasets," International Journal of Robotics Research, 2016.（EuRoC 資料集的官方說明，本章 26.6 節依據）
9. D. Schubert et al., "The TUM VI Benchmark for Evaluating Visual-Inertial Odometry," IROS, 2018.（TUM-VI 資料集，含光度校正與硬體時間同步的設計說明）
10. C. Campos et al., "ORB-SLAM3," IEEE T-RO, 2021.（第 23 章的系統，本章實作的主要工具）
11. T. Qin, P. Li, and S. Shen, "VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator," IEEE T-RO, 2018.（無人機視覺慣性系統的另一代表作）
12. D. Scaramuzza and Z. Zhang, "Visual-Inertial Odometry of Aerial Robots," Encyclopedia of Robotics, Springer, 2019.（空中機器人視覺慣性導航的權威綜述，UZH 研究群，呼應第 2 章）
13. K. Sun et al., "Robust Stereo Visual Inertial Odometry for Fast Autonomous Flight," IEEE RA-L, 2018.（高速飛行 VIO 的代表性工作，案例 26-3 提及）
14. PX4 與 ArduPilot 開源飛控之視覺定位整合文件.（把 VIO 輸出接入飛控的實務指引，實體飛行必讀）

小結

8. 無人機定位有四項特殊困難：完整六自由度運動、沒有里程計（只能靠會漂移的 IMU）、速度快且動態劇烈、以及失誤後果嚴重；加上載重電力限制與嚴苛的即時性要求，使它成為最難的 SLAM 平台之一。

9. GPS 拒止場景包含室內、橋下、隧道、都市峽谷、植被下與人為干擾；可用手段依序為 IMU、氣壓計、光流、測距、VIO、VI-SLAM 與 UWB / 視覺標記，實務採分層架構並讓失效逐級降級而非瞬間崩潰。
10. 三種視覺定位技術的層次：光流只給水平速度但運算極省，支撐定點懸停；VIO 給完整 6-DoF 位姿但會漂移；VI-SLAM 加上地圖與閉環可修正長期漂移。應依任務需求選擇，不用超過需求的技術。
11. 高速飛行造成三重打擊：動態模糊損害特徵追蹤、特徵軌跡縮短使中期關聯難以建立、控制迴路延遲惡化；對策包含提高快門速度、使用全域快門、以 IMU 預測改善追蹤、影像增強、以及自適應調整視覺與慣性的權重。
12. 室內飛行實驗的安全原則不可妥協：防護網、微型平台、緊急停止、分階段測試；場地需注意紋理、光線與氣流；實驗流程應由懸停測試逐步進展到挑戰測試。
13. EuRoC 提供十一段微型飛行器序列、雙目影像與動作捕捉真值，速度很少超過每秒 5 公尺；TUM-VI 則提供 1024×1024 影像、光度校正與硬體時間同步，場景多樣性較高。比較數據時須留意各研究的評測條件。
14. 無人機自主可分穩定、自主、智慧三個層次；技術演進沿運算下沉、感測輕量化、智慧上升三條主線推進，而三者正在交會——小、省電且具語意理解能力的無人機正在成為現實。

承先啟後

本章把飛行帶來的三個新問題量化了：漂移方面，72 秒的飛行後半段，光流的平均誤差是 0.452 公尺、VIO 0.324 公尺、VI-SLAM 0.041 公尺，而真正的差別在形狀——只有 VI-SLAM 的誤差不隨時間成長；速度方面，特徵匹配率從每秒 3 公尺的 100% 掉到 28 公尺的 31%，而更致命的是特徵停留在視野中的幀數從 104 幀掉到 1 幀，最有效的對策是快門（17 m/s 下由 1/125 提到 1/500，匹配率從 23% 升到 79%）；震動方面，混疊會讓螺旋

槳的高頻振動偽裝成 1.00 公尺 / 秒² 的假零偏（4 秒即造成 8 公尺的位置誤差），而且提高取樣率無效，只有取樣前的機械減震能把它壓到 0.023。

本章完成了單機無人機導航的討論。但無論多先進，單一機器人的視野、續航與運算能力終究有限。下一章我們把視野擴大到群體：多台機器人如何協同建圖？各自的地圖如何融合成一致的全域地圖？雲端與邊緣如何分工？通訊頻寬、時間同步與資料安全又有哪些課題？第 27 章將完成第七篇的最後一塊拼圖，也為第八篇的實作與第九篇的 AI 語意導航鋪路。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 27 章

多機器人 SLAM 與雲端 / 邊緣協作

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程　/　高中 AI 機器人社團課程

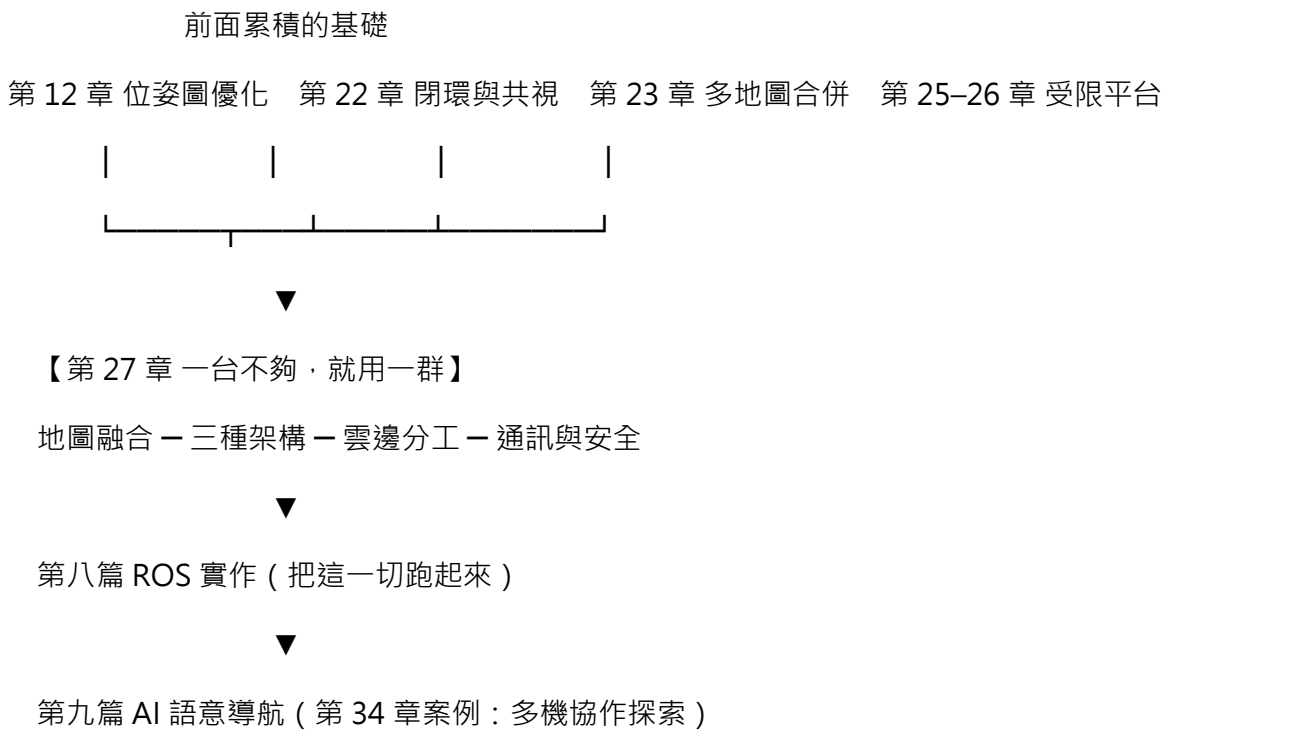
本章在全書的位置

第 25 章把機器人做小，第 26 章讓它飛起來。兩章的共同結論是：單一平台的能力終究有限。本章的答案不是把它做得更強，而是多來幾台。

但「多來幾台」不是把一台的能力乘以 N。多機器人系統有它自己的問題——地圖怎麼對齊、誰跟誰說話、頻寬不夠時要先傳什麼、以及一台機器人認錯地方時會不會拖垮其他所有機器人。這些問題在單機系統裡完全不存在。

本章的三張圖分別量化其中三件事：圖 27-1 算出多台機器人協同探索到底能快多少，以及「共享地圖」這件事值多少；圖 27-2 量出一條錯誤的跨機器人閉環能造成多大破壞；圖 27-3 則回答一個很實際的問題——頻寬只夠傳幾條閉環時，該傳哪幾條。

圖 27-0　第 27 章的連結地圖



學習目標

前面二十五章討論的都是「一台機器人」。但單機的視野、續航與運算終究有限：一台機器狗要花數小時才能巡完整棟系館，而十台可能只需二十分鐘。本章把視野擴大到群體，也完

成第七篇的最後一塊拼圖。學完本章之後，同學應能夠達成下列目標：

15. 說明多機器人 SLAM 相對單機的優勢與新增的三大挑戰。
16. 說明地圖融合的流程與跨機器人閉環偵測的關鍵。
17. 比較集中式、分散式與去中心化三種協作架構的優缺點。
18. 說明雲端 SLAM 與邊緣 SLAM 的分工原則與適用場合。
19. 規劃一個多台機器狗校園巡檢的系統架構與任務分配。
20. 說明 SLAM 與 AI Agent 如何協作，以及 VLM 在多機協調中的角色。
21. 說明多機器人系統在隊形、通訊、資料同步與安全上的課題。

核心概念

中文名詞	英文名詞	概念說明
協作式 SLAM	Collaborative SLAM	多台機器人共同建立一致地圖的技術。
集中式架構	Centralized	所有資料匯集到中央伺服器處理。
分散式架構	Distributed	運算分散在多個節點，彼此交換資訊。
去中心化	Decentralized	無中央節點，機器人直接彼此通訊並各自融合。
地圖融合	Map Fusion / Merging	把多張地圖對齊並合併為一致的全域地圖。
跨機器人閉環	Inter-robot Loop Closure	不同機器人觀測到同一地點所建立的約束。
相對變換	Relative Transform	兩台機器人座標系之間的旋轉與平移關係。
會合	Rendezvous	兩台機器人相遇並交換資訊的事件。
感知混淆	Perceptual Aliasing	不同地點看起來相同而造成的錯誤關聯。
離群點剔除	Outlier Rejection	排除錯誤閉環以免破壞全域地圖。
邊緣運算	Edge Computing	在靠近機器人的本地節點進行運算。

中文名詞	英文名詞	概念說明
雲端運算	Cloud Computing	在遠端資料中心進行運算與儲存。
異質團隊	Heterogeneous Team	由不同類型機器人（地面、空中）組成的團隊。

27.1　單機 SLAM 與多機 SLAM

本節說明為什麼要用多台機器人，以及這個選擇帶來什麼新問題。

多機器人的四項優勢

- 速度：多台分頭探索，完成大範圍任務的時間大幅縮短。這是最直接的動機——第 24 章案例 24-3 的多機器人協作探索正是這個思路。
- 容錯：單台故障不影響整體任務。相較之下，單一昂貴機器人一旦故障，整個任務就中止。
- 視角互補：從多個位置同時觀測，能建立更完整的環境模型，也能減少遮蔽造成的盲區。
- 能力互補：異質團隊能結合不同平台的長處——地面機器人續航長、承載大；無人機視野廣、能到高處（第 26.7 節提過的地空協作）。

三大新挑戰

但多機器人不是「單機乘以 N」這麼簡單，它帶來三個單機不存在的問題：

60. 座標系統一：每台機器人開機時各自以自己的起點為原點（第 10 章）。要把它們的地圖合在一起，必須先知道彼此座標系之間的相對變換——而這個變換一開始是未知的。
61. 資料關聯的擴大：單機 SLAM 只需判斷「我現在看到的，是不是我以前看過的」（第 23 章的長期關聯）；多機器人還要判斷「我看到的，是不是別人看過的」。搜尋空間變大，錯誤關聯的風險也隨之上升。
62. 通訊限制：機器人之間的頻寬有限、連線可能中斷、延遲不固定。所有協作機制都必須在這個現實下運作——這是本章反覆出現的主軸。

與單機技術的關係

好消息是：多機器人 SLAM 的核心技術，同學都已經學過了。

63. 每台機器人的自身定位建圖：第 13 至 26 章的各種單機 SLAM。

64. 跨機器人閉環偵測：第 20、22 章的地點辨識（詞袋模型）與幾何驗證。

65. 地圖融合的最佳化：第 12 章的位姿圖優化。

66. 錯誤閉環的處理：第 12、15 章的穩健核函數與離群點剔除。

換句話說，多機器人 SLAM 是把單機的技術「擴充到多個座標系與多個資料來源」。理解這一點，同學就不會覺得本章的內容陌生——它是前面所有章節的自然延伸。

量一量：多幾台到底能快多少

27.1 節列出了多機器人 SLAM 的四項優勢，第一項是「速度」。這一項聽起來最理所當然，卻也最容易被高估。本小節把它算出來。

實驗設定是一層 34×18 公尺的系館樓層（中央走廊加兩側十二個房間）。N 台機器人從走廊上不同位置出發，各自以「前往最近的前沿」策略探索，感測半徑 3 公尺，直到自由空間被探索 98.5% 為止。兩種設定：共享同一張地圖（看得到別人探過哪裡），以及各自建圖、完全不交換資訊。

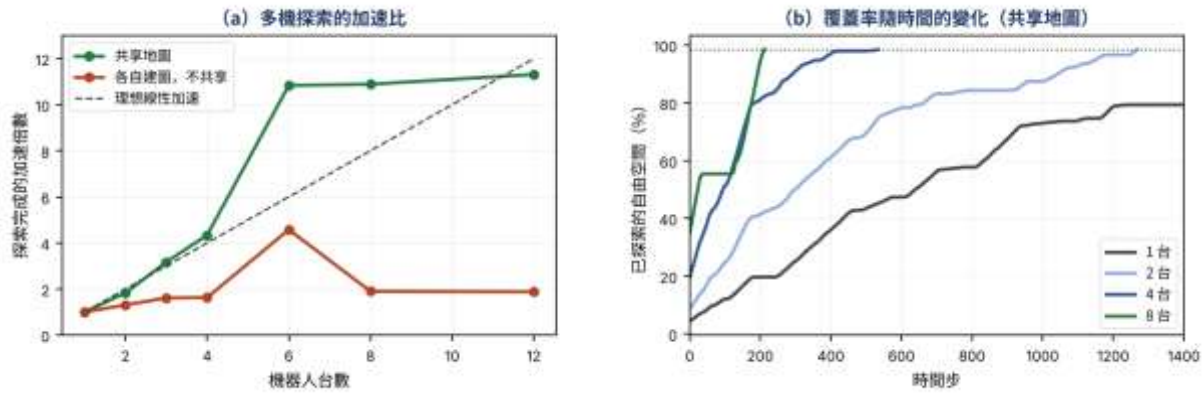


圖 27-1　多機器人協同探索的加速比

（a）是加速比。共享地圖時，兩台是 1.83 倍、三台 3.17 倍、四台 4.33 倍、六台 10.8 倍，之後就停住了——八台 10.9 倍、十二台 11.3 倍。也就是說，這層樓的「平行度上限」大約在六台。再多的機器人不是沒用，而是它們抵達最後幾個未探索角落所需的移動時間，已經

成為無法再壓縮的瓶頸。

三台與四台出現略高於線性的加速（3.17 與 4.33），原因是起點分散在走廊各處，省下了單機必須從頭走到尾的那段路程。這提醒我們：多機系統的效益有一部分來自「分散部署」，而不只是「平行運算」。

真正值得注意的是紅線。不共享地圖時，兩台只有 1.31 倍、四台 1.63 倍，而且曲線上下起伏不定（六台 4.57 倍，八台卻掉回 1.90 倍）。這個不穩定本身就是結論：沒有共享地圖的多機器人會重複探索彼此已經走過的區域，最終表現完全取決於它們碰巧從哪裡出發。以八台為例，共享地圖讓完成時間縮短了 83%。

這就是為什麼 27.2 節的地圖融合是整章的核心操作。沒有融合，多機器人只是多台各自為政的單機；有了融合，它們才真正成為一個系統。

加速比飽和之後，多出來的機器人做什麼

圖 27-1(a) 顯示六台以上就不再變快。但這不代表第七台沒有價值。

27.1 節提到的另外三項優勢——容錯、視角多樣性與任務分工——都不隨台數飽和。第七台可以在某一台故障時接手，可以從不同角度觀測同一個物件以提升語意判讀的可靠度（第 32 章），也可以專門負責充當通訊中繼。

所以規劃機群規模時，應該分開問兩個問題：「要多快完成」決定探索所需的台數，「要多可靠」與「要做哪些事」決定額外的台數。

27.2　Map Fusion 地圖融合

地圖融合是多機器人 SLAM 的核心操作：把兩張（或多張）各自建立的地圖，對齊並合併成一致的全域地圖。

與單機閉環的異同

同學應該立刻聯想到第 23.6 節的 Atlas 地圖合併。兩者的數學本質相同——找出兩張地圖之間的相對變換，再以位姿圖優化把修正傳播出去。差別在於：

67. 單機的多地圖：兩張地圖來自同一台機器人的不同時段，感測器與參數完全相同。

68. 多機器人的地圖融合：兩張地圖來自不同機器人，可能使用不同的感測器、不同的視角、甚至不同的地圖表示。異質團隊的融合尤其困難。

融合的三步驟

69. 偵測相遇：判斷兩台機器人是否觀測到同一個地點。這可以是實體的會合（兩台機器人互相看見），也可以是「地點辨識的重疊」——A 機器人現在看到的場景，B 機器人稍早也看過。

70. 估計相對變換：確認重疊後，由共同觀測到的特徵或掃描，解出兩台機器人座標系之間的旋轉與平移（第 21 章的幾何求解，或第 17 章的點雲配準）。

71. 融合與最佳化：把兩張位姿圖用這條「跨機器人的邊」連起來，執行位姿圖優化（第 12 章），使整個系統達成全域一致。

關鍵風險：錯誤的跨機器人閉環

第 12、15、22 章都強調過：一條錯誤的閉環會把地圖硬拉到錯誤位置。在多機器人系統中，這個風險更高，因為：

72. 候選空間更大：要比對的不只是自己的歷史，還有所有其他機器人的觀測。

73. 感知混淆更嚴重：不同機器人可能在不同的相似地點（例如兩間格局相同的教室），產生看似正確的錯誤匹配。

74. 後果更嚴重：一條錯誤的跨機器人邊，會同時破壞多台機器人的地圖。

因此穩健的離群點剔除是多機器人 SLAM 的必備元件。相關系統會整合穩健的離群點剔除機制來強化這個做法，並使其適應零星的機器人間通訊。另有系統採用分散式的漸進非凸化演算法，並把語意資料納入產出的地圖中。

通訊效率的考量

地圖融合需要交換資料，而頻寬有限。因此如何「用最少的資料達成最好的融合」，是這個領域的核心課題。有系統引入基於代數連通度最大化的新穎跨機器人閉環優先排序技術，以減少通訊並加速收斂；它也採用去中心化的鄰居管理與位姿圖優化方式，適合零星的機器人間通訊，並支援不同感測器類型的描述子——影像用 CosPlace 與 NetVLAD、光達掃描用 Scan Context。

這裡的核心洞見是：不是所有閉環候選都同等重要。有些閉環對改善全域一致性貢獻很大，有些則影響甚微。優先傳輸高價值的候選，能在有限頻寬下獲得最大效益——這是一個極具工程智慧的思路。

量一量：一條錯誤的跨機邊有多危險

27.2 節說「一條錯誤的跨機器人邊，會同時破壞多台機器人的地圖」。這句話值得用數字說一次，因為它的嚴重程度超過多數人的直覺。

實驗設定：三台機器人各走一條部分重疊的環形路線，每台 50 個關鍵幀，里程計有雜訊。它們之間有 42 條正確的跨機器人閉環。接著刻意混入若干條「感知混淆」造成的錯誤邊——機器人以為看到的是對方的第 j 幀，實際上那是長得很像的另一處。比較一般最小平方與穩健核（漸進非凸化）兩種求解方式。

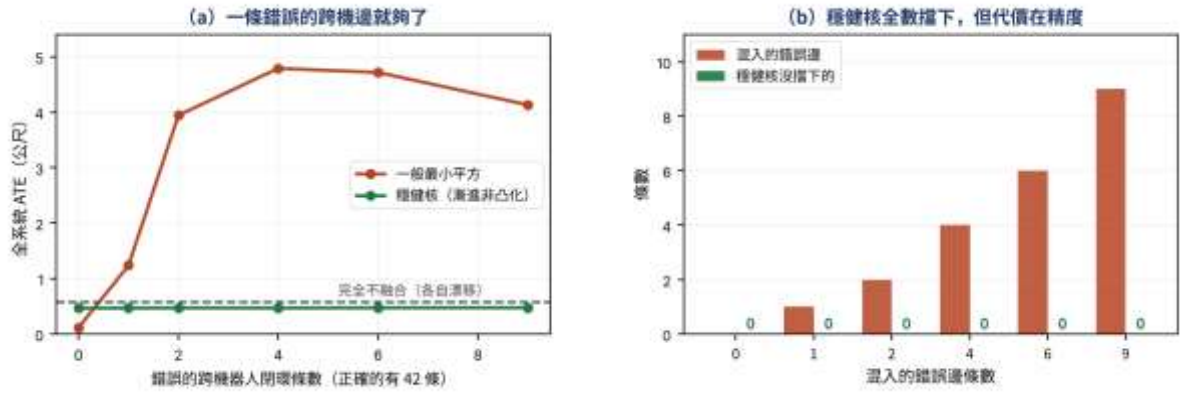


圖 27-2　錯誤的跨機器人閉環與穩健剔除

（a）的紅線是重點。沒有錯誤邊時，融合後的全系統 ATE 是 0.119 公尺，比完全不融合的

0.576 公尺好了近五倍——這是地圖融合的正面價值。但只要混入一條錯誤邊，ATE 就跳到 1.238 公尺；兩條是 3.946 公尺；四條以上停在約 4.8 公尺。

請注意一條錯誤邊造成的破壞：從 0.119 跳到 1.238，放大了十倍以上，而且比完全不融合還糟兩倍。換句話說，在這個情境下，「融合但混進一條錯誤邊」比「根本不融合」更差。這就是 27.2 節把離群點剔除列為「必備元件」的理由——它不是用來提升精度的，是用來避免災難的。

綠線是加上穩健核之後。不論混入幾條錯誤邊（測到九條），ATE 都穩定在 0.47 公尺左右，完全不隨錯誤邊增加而惡化。（b）顯示它的篩選能力：所有錯誤邊都被擋下，沒有一條漏網。

但這裡有一個必須誠實說明的代價。即使在完全沒有錯誤邊的情況下，穩健核的 ATE 也是 0.469 公尺，而一般最小平方是 0.119 公尺——差了四倍。原因是穩健核會降低「所有」大殘差的權重，包括那些只是雜訊比較大的正確邊。它用精度換取了對離群點的免疫力。

實務上的處理方式是兩階段：先用穩健核判別出內點集合，再只用內點重做一次一般最小平方。這樣理論上能同時拿到兩者的好處，但判別門檻要訂得很小心——門檻一鬆就會放進錯誤邊，而放進一條就前功盡棄。這是多機器人 SLAM 實作中最需要調校、也最容易出事的地方。

量一量：頻寬不夠時該傳哪幾條

27.2 節最後提到「不是所有閉環候選都同等重要」，並介紹了以代數連通度最大化來排序候選的做法。這個想法相當抽象，本小節用實驗讓它變得具體。

代數連通度是圖的拉普拉斯矩陣第二小的特徵值，直覺上衡量「這張圖連得有多緊」——值越大，表示圖越不容易被切成兩半，誤差也越容易在整張圖上重新分配（第 22 章圖 22-2 的道理）。實驗比較兩種挑選策略：隨機挑 k 條候選，以及貪婪地每次挑「最能提升代數連通度」的那一條。

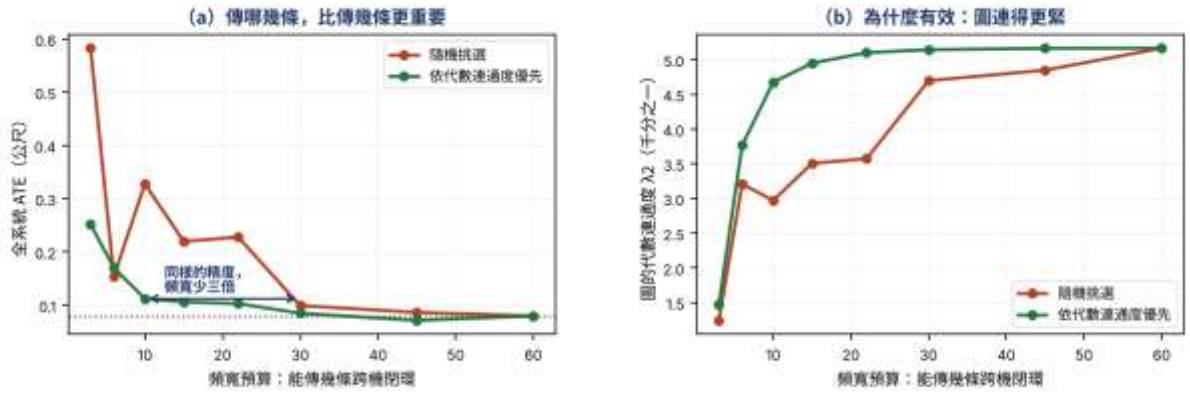


圖 27-3　有限頻寬下，該優先傳哪些閉環候選

（a）是結果。預算只有 3 條時，隨機挑選的 ATE 是 0.583 公尺，依連通度排序是 0.252 公尺；預算 10 條時分別是 0.328 與 0.112 公尺；預算 22 條時是 0.228 與 0.103 公尺。在中低預算下，排序讓誤差減半到減三分之二。

更有用的讀法是橫著看：依連通度排序只要 10 條就能達到 0.112 公尺，而隨機挑選要到 30 條才達到相近的水準。也就是說，光是「挑對要傳哪幾條」，就能把頻寬需求降到三分之一。對於只有零星通訊機會的機器人團隊，這個差別是決定性的。

（b）解釋為什麼有效。縱軸是圖的代數連通度，綠線在每個預算下都高於紅線——排序策略用同樣的邊數換到了更緊密的圖結構。兩條線在預算 60（全部傳完）時重合，因為此時兩者挑的是同一個集合。

這個實驗也示範了一個通用的工程思路：當資源有限時，與其問「能不能多拿一點資源」，不如先問「現有的資源有沒有花在刀口上」。同樣的思路在第 22 章的關鍵幀剔除、第 25 章的事件驅動運算都出現過。

代數連通度為什麼是對的指標

直覺上我們會想挑「最確定的閉環」或「最近的閉環」。但那些閉環往往連接的是已經靠得很近的兩個節點——資訊是重複的，對全域一致性幫助不大。

代數連通度抓到的是另一件事：這條邊有沒有把圖上原本相距很遠的兩塊連起來。把

兩塊鬆散的區域縫在一起，遠比在已經很緊的地方再加一針有用。

這與第 22 章「共視邊讓修正量傳遍整條軌跡」是同一個道理，只是從定性的觀察變成了一個可以計算、可以排序的數字。

27.3　Collaborative SLAM 的三種架構

多機器人 SLAM 的系統架構，是本章最重要的設計決策。多機器人 SLAM 架構的光譜，從集中式一路到完全去中心化。

一、集中式架構

所有機器人把資料傳給一台中央伺服器，由伺服器統一處理與最佳化，再把結果分發回去。

像 CCM-SLAM 這類集中式系統把資料匯集到伺服器進行聯合最佳化；這簡化了地圖融合，但引入了通訊瓶頸與單點故障。

75. 優點：地圖融合簡單（所有資料都在同一處）、全域最佳化容易、機器人端運算負擔輕。
76. 缺點：通訊瓶頸（所有資料都要傳）、單點故障（伺服器一掛全系統停擺）、擴展性差。
77. 代表系統：CCM-SLAM、COVINS（針對視覺慣性感測、以移除冗餘關鍵幀來稀疏化）、LAMP 2.0（光達系統，以圖神經網路預測每個跨機器人閉環候選的優先順序）。

二、分散式與去中心化架構

在分散式系統中，處理分散在多個節點或代理上；機器人蒐集資料並執行自己的定位與建圖，同時透過持續通訊彼此分享資訊，以建立完整的全域地圖。

去中心化的做法中，機器人直接彼此交換資訊，並在機載完成所有資料融合，不依賴中央實體來協調系統。

78. 優點：無單點故障、擴展性佳、對通訊中斷較有韌性。

79. 缺點：同步複雜度高、需要分散式最佳化演算法、每台機器人的運算負擔較重。

80. 代表系統：DOOR-SLAM 與 Kimera-Multi 執行點對點的位姿圖優化並具離群點剔除，以增加擴展性為代價換來更高的同步複雜度；Swarm-SLAM 提供了支援多種感測模態的 ROS 2 去中心化框架；LAMP 與 LAMP 2.0 則處理 DARPA 地下挑戰賽中的地下多機器人 SLAM（第 1 章案例 1-2）。

架構選擇的關鍵：通訊條件

集中式架構的缺點在於，它們需要與共同後端保持良好的通訊，而若該節點失效，整個系統就會失效；此外它們無法擴展到大型機器人團隊。

因此，只依賴機器人之間零星、間歇通訊的去中心化解法，更適合大範圍的場景。

架構	運算位置	優點	缺點	適用場景
集中式	中央伺服器	融合簡單、機器人端輕量	通訊瓶頸、單點故障	通訊良好的小型團隊
分散式	多節點協同	無單點故障、擴展性佳	同步複雜	中大型團隊
去中心化	各機器人機載	對通訊中斷有韌性	機載運算負擔重	通訊零星的大範圍任務

表 27-1　三種協作 SLAM 架構的比較

實務上的混合

真實系統往往不是純粹的某一種。例如：機器人在機載執行自己的 SLAM（去中心化的部分），但當通訊良好時把關鍵幀傳給伺服器做全域最佳化（集中式的部分）。這種混合設

計能兼顧兩者的優點，也是本書反覆強調的工程思維——不必在教科書式的分類中二選一。

27.4　Cloud SLAM 與 Edge SLAM

上一節談的是「機器人之間」如何分工，本節談的是「機器人與外部運算資源」如何分工。

三層運算的分工

層級	位置	延遲	運算能力	適合的任務
機載	機器人本身	極低	受限（第 25 章）	即時控制、姿態估計、避障
邊緣	本地伺服器 / 基地台	低（數毫秒至數十毫秒）	中至高	SLAM 後端、地圖融合、AI 推論
雲端	遠端資料中心	高（數十至數百毫秒）	極高	大規模地圖儲存、模型訓練、跨場域分析

表 27-2　機載、邊緣、雲端三層運算的分工

分工的基本原則

決定「什麼該在哪一層做」，有三個判準：

81. 即時性要求：控制迴路必須在機載（第 26.1 節的無人機案例最為明顯）；可以容忍數十毫秒延遲的任務才適合送到邊緣。
82. 資料量：傳輸大量原始資料成本高。若能在機載先做壓縮或特徵提取，再傳輸精簡後的資料，效率會好得多（這正是 27.2 節通訊效率的思路）。
83. 安全與隱私：涉及個人隱私的影像資料，是否適合傳上雲端？這在醫院、學校等場域是必須考慮的問題。

邊緣輔助的協作 SLAM

邊緣運算特別適合多機器人 SLAM，因為它同時解決了兩個問題：機器人端運算不足、以及集中式架構的延遲問題。

有系統以卸載繁重運算來最佳化伺服器端，同時以機載的視覺里程計維持代理的自主性。這是一個典型的分工：機器人自己做即時的前端（不依賴網路），把耗時的後端最佳化與地圖融合交給邊緣伺服器。

通訊技術的進步也在推動這個方向。有研究運用 5G 通訊以處理來自多台機器人的即時稠密體積資料，顯示高吞吐與低延遲如何讓大型感測器串流的卸載成為可能。第 5.4 節提過 5G 在機器人定位的應用，此處則是它在協作運算上的價值。

通訊品質作為規劃資源

一個近年重要的觀念轉變值得同學注意。有研究的核心想法是把通訊品質視為一種規劃資源，並結合多代理協作，使「導航斷崖」成為可管理的降級事件，而非硬性的運作極限。

這個思路極具啟發性：與其被動接受「這裡沒訊號」，不如主動把通訊品質納入路徑規劃——例如讓一台機器人停在訊號良好處作為中繼、或安排機器人定期回到通訊區交換資料。這與第 26.2 節「讓失效逐級降級」的設計哲學一脈相承。

27.5　多台機器狗校園巡檢

本節把前面的技術整合成一個具體的系統規劃，這也是第 31 章專題四的基礎。

為什麼需要多台

以一般大學的校園規模估算：一台機器狗以每秒 0.5 公尺的巡檢速度，要走遍一棟系館的所有走廊與樓層，可能需要數小時，而電池續航往往不足一小時。多台機器狗分頭進行，才能在單次充電內完成任務。

系統架構

圖 27-4　多機器狗校園巡檢的系統架構

【機載層】每台機器狗

- 光達慣性 SLAM（第 18 章）或視覺慣性 SLAM（第 23 章）
- 即時避障與步態控制
- 語意感知：YOLO 偵測巡檢目標（第 24 章）

↓ 傳送：關鍵幀描述子、位姿圖、巡檢發現

【邊緣層】校園內的本地伺服器

- 跨機器狗閉環偵測與地圖融合（27.2 節）
- 全域位姿圖優化（第 12 章）
- 任務分配與路線協調

↓ 傳送：精簡後的地圖、巡檢報告

【雲端層】（選用）

- 長期地圖儲存與跨時間比對（變化偵測）
- AI 模型訓練與更新
- 管理介面與報表

任務分配策略

84. 依區域劃分：最簡單——每台機器狗負責一個樓層或一個區域。優點是互不干擾、實作容易；缺點是無法動態調整，若某區提早完成，該機器狗就閒置。
85. 依邊界動態分配：第 24 章案例 24-3 提過的做法——把已探索與未探索的交界（邊界）動態分配給最合適的機器人。效率較高，但需要即時的協調機制。
86. 依能力分配：異質團隊的做法——機器狗負責樓梯與室內，無人機負責中庭與屋頂，輪型機器人負責平坦的長廊（續航最佳）。

實務課題

87. 起始座標對齊：讓所有機器狗從同一個充電站出發，或在起點放置 AprilTag（第 24.3 節），即可直接建立共同的座標系，大幅簡化地圖融合。這是一個「用簡單方法避開困難問題」的好例子。

88. 通訊覆蓋：校園的 Wi-Fi 未必處處覆蓋（地下室、樓梯間）。系統必須能容忍斷線，並在恢復連線時補傳資料——這正是 27.3 節去中心化架構的價值。

89. 安全與互相避讓：多台機器狗可能在走廊相遇，需要協調通行。此外還要考慮與行人的互動（第 24 章的社交感知導航）。

90. 電池管理：機器狗續航短，系統需規劃充電輪替——例如兩台巡檢、一台充電，輪流交替以維持持續覆蓋。

27.6　SLAM 與 AI Agent 協作

第 24.5 節介紹過 VLM 輔助導航，本節聚焦 AI 在多機器人協調中的角色。

VLM 作為全域規劃器

第 24 章案例 24-3 已詳述過這個方向：VLM 不做低階控制，而是扮演「指揮官」——看著全域地圖，運用常識推理判斷哪些方向最可能找到目標，再把任務分配給不同的機器人。

這個分工的優雅之處在於：VLM 的優勢是語意理解與常識推理（慢但聰明），機器人的優勢是即時感知與控制（快但不理解意義）。讓各自做擅長的事，是最合理的架構。

AI Agent 在多機系統中的三種角色

22. 任務規劃者：把高階目標（「巡檢三樓所有教室」）分解為具體的子任務，並分配給各機器人。這需要理解場域的語意結構（語意地圖，第 3、24 章）。

23. 異常判讀者：巡檢機器人回傳大量影像，由人工逐一檢視不切實際。AI Agent 能篩選出「需要人類注意」的異常（裂縫、漏水、堵塞的通道），大幅減輕人力負擔。

24. 協調仲裁者：當多台機器人的需求衝突（都想去同一區域、都需要充電），Agent 依全域目標做出決策。

語意地圖的共享

多機器人協作的一個進階方向，是共享語意而非只共享幾何。有系統把語意資料納入產出的地圖中，能在僅依賴局部通訊的情況下，即時建立全域一致的度量語意三維網格模型，並對錯誤的閉環具有韌性。

共享語意的價值在於：機器人 A 辨識出「這是電梯」之後，機器人 B 不需要重新辨識，直接沿用即可。這不只省運算，也讓整個團隊對環境的理解保持一致。

務實的提醒

第 24.5 節強調過的限制，在多機器人系統中同樣適用甚至更嚴重：

91. 延遲：VLM 推論耗時，若每個決策都要等它，系統反應會過慢。因此 AI 應處理「秒級以上」的決策，即時避讓仍由傳統方法負責。
92. 幻覺：AI 判斷錯誤可能導致整個團隊往錯誤方向探索。系統應保留幾何層的驗證與人類的監督介面。
93. 通訊：AI 決策通常在邊緣或雲端執行，這意味著它依賴通訊。斷線時，機器人必須能退回預設的自主行為（27.4 節的降級思維）。

27.7　隊形、通訊、資料同步與安全

本節收攏多機器人系統的四項工程課題，它們雖不像演算法那樣光鮮，卻往往決定系統能否真正運作。

一、隊形與空間協調

94. 避免碰撞：多台機器人在同一空間移動，必須避免互撞。做法包括共享即時位置、劃分行進區域、以及在狹窄處（走廊、門口）設定通行規則。

95. 維持通訊鏈：若團隊需要保持連線，隊形必須考慮通訊範圍。有時會刻意讓一台機器人停在中繼位置（27.4 節提過的把通訊視為規劃資源）。

96. 覆蓋效率：隊形也影響探索效率。分散能加快覆蓋，但過度分散會失去互相支援與通訊的能力。

二、通訊

97. 頻寬限制：原始影像與點雲的資料量極大，不可能全部傳輸。實務上傳輸的是精簡的表示——關鍵幀描述子、位姿圖、稀疏特徵（27.2 節的效率考量）。

98. 間歇連線：實際場域中，連線會斷。系統設計必須假設「隨時可能斷線」，並在恢復時能補傳與重新同步。

99. 通訊拓樸：是所有機器人都能互相通訊，還是只有相鄰的能？這直接影響架構選擇（27.3 節）。

三、資料同步

100. 時間同步：這是多機器人系統最容易被低估的難題。每台機器人的時鐘略有差異，若不校準，融合時會出現時間錯位。第 5、10 章反覆強調過時間同步的重要性，在多機系統中影響更大。常用做法是網路時間協定或以 GNSS 授時（第 5.1 節提過 GNSS 同時提供精確授時）。

101. 座標同步：即 27.1 節提過的座標系統一問題。除了靠地圖融合估計，也可用共同的起始點或環境中的已知標記（27.5 節的實務做法）。

102. 版本一致：當地圖被更新（例如閉環修正後），所有機器人都需要拿到最新版本。若某台機器人用著舊地圖行動，可能導致衝突。

四、安全

103. 通訊安全：機器人之間的通訊若被竊聽或竄改，攻擊者可能注入錯誤的地圖資料，誤導整個團隊。加密與身分驗證是必要的。

104. 資料隱私：巡檢機器人拍攝的影像可能包含人臉與私人空間。在校園、醫院等場域，必須明確規範資料的蒐集、儲存與存取權限。這不只是技術問題，也是倫理與法規問題。

105. 實體安全：多台機器人同時運作，對周圍人員的風險增加。系統需有全體緊急停止機制，且每台機器人都應有獨立的安全底線（即使失去通訊也不會失控）。

106. 故障隔離：一台機器人故障或行為異常時，系統應能將其隔離，避免它的錯誤資料污染全域地圖——這正是 27.2 節離群點剔除在系統層級的延伸。

第七篇的收束

第七篇從第 25 章的毫瓦級微型平台出發，經過第 26 章的無人機導航，到本章的群體協作，呈現了一條清晰的線索：當單一機器人的能力受限時，答案不是把它做得更大更強，而是讓它更小、更省電、然後用更多台協同工作。

這個思路與自然界的策略不謀而合——螞蟻與蜜蜂個體能力有限，群體卻能完成驚人的任務。本篇介紹的技術，正是讓機器人團隊具備這種集體智慧的基礎。

下一篇，我們回到最實際的層面：如何把這一切真正跑起來。第八篇「ROS / Python / C++ 實作」將帶同學建置開發環境、實作 2D 光達建圖與導航、執行視覺 SLAM，並完成完整的專題。理論走到這裡，是時候讓機器人真的動起來了。

27.8　多機器人 SLAM 的三個實務邊界

本章量了三件事，每一件都指向一個實務上的邊界。本節把它們收攏起來。

一、台數的邊界：由場域決定，不由預算決定

圖 27-1 顯示加速比在六台左右飽和。這個數字不是普適的——它由場域的大小、拓樸複雜度與感測半徑共同決定。同學在規劃機群時，應該先用自己的場域跑一次圖 27-1 的模擬，找出屬於自己的飽和點，再決定要買幾台。

一個粗略的估計方式是：把場域的總路徑長度除以「單台在續航時間內能走的距離」，得到的就是「至少需要幾台」；而飽和點通常落在這個數字的一到兩倍之間。超過飽和點之後增加的台數，價值在容錯與分工，不在速度。

二、信任的邊界：寧可少融合，不可錯融合

圖 27-2 給出的數字很極端：一條錯誤邊讓系統比完全不融合還糟兩倍。這個不對稱決定了多機器人系統的設計原則——融合的門檻應該訂得比單機閉環更嚴格。

具體的做法包括：要求更多的內點數才接受跨機匹配、要求連續多幀都偵測到同一個匹配、以及在接受之前先用幾何一致性檢查（例如三台機器人之間的變換要能首尾相接回到原點）。這些檢查都會讓系統錯過一些正確的閉環，但依圖 27-2 的數字，這個代價遠小於接受一條錯誤邊。

三、頻寬的邊界：先排序，再壓縮

面對頻寬限制，直覺的做法是壓縮資料——降低影像解析度、減少特徵點數、稀疏化關鍵幀。圖 27-3 指出另一個常被忽略的維度：挑選。在傳同樣多資料的前提下，挑對要傳的內容可以讓效果提升兩到三倍。

務實的順序是：先排序（決定傳哪些），再壓縮（決定每一筆傳多細）。順序顛倒的話，你會把頻寬平均花在所有候選上，結果每一條都傳得不夠清楚，反而都用不上。

	本章量到的數字	對設計的意義
台數	六台飽和於 11 倍加速	飽和點由場域決定，先模擬再採購
共享地圖	八台時縮短 83% 時間	地圖融合是多機系統的前提，不是加分項
錯誤閉環	一條就讓 ATE 由 0.119 升到 1.238 公尺	融合門檻要比單機閉環更嚴

穩健核	全數擋下，但無錯誤時精度由 0.119 降為 0.469	保險有保費，兩階段處理可兼顧
候選排序	10 條排序過 ≒ 30 條隨機	先排序再壓縮

表 27-3　本章三個實驗的設計意涵

公式或流程

一、跨機器人的座標變換

設機器人 A 與 B 各自的世界座標系為 W_A、W_B。地圖融合的核心，是求出兩者之間的相對變換：

$$T_{W_A \leftarrow W_B}$$

取得這個變換後，B 的任一位姿即可轉換到 A 的座標系：

$$p_{W_A} = T_{W_A \leftarrow W_B} \cdot p_{W_B}$$

這正是第 10 章「變換組合就是矩陣連乘」在多機器人上的應用。若團隊有 N 台機器人，原則上只需 N - 1 個相對變換就能把所有地圖統一到同一座標系（構成一棵生成樹，呼應第 22.6 節的概念）。

二、多機器人位姿圖的目標函數

把所有機器人的位姿圖合併後，整體最佳化的目標為：

$$\min \Sigma_{機器人內部邊} \|e\|^2_\Sigma + \Sigma_{跨機器人邊} \rho(\|e\|^2_\Sigma)$$

第一項是各機器人自己的里程計與閉環約束，第二項是跨機器人的閉環約束。

請特別注意第二項外面的穩健核函數 ρ——跨機器人邊的錯誤風險遠高於機器人內部的邊（27.2 節），因此必須用穩健核函數保護。這個細節體現了「依可信度差異化處理」的思想，與第 11 章卡爾曼增益、第 12 章資訊矩陣一脈相承。

三、通訊量的估算

多機器人系統的通訊需求，可粗估為：

$$\text{總通訊量} \approx N \times (N-1) \times \text{每對機器人交換的資料量} \times \text{頻率}$$

注意 N 的平方成長——這正是集中式與全連通架構難以擴展的數學原因。以 10 台機器人為例，兩兩通訊的配對數是 45 對；若增加到 20 台，配對數躍升至 190 對。

這條式子解釋了 27.2 節「閉環候選優先排序」與 27.3 節「去中心化架構」的必要性：當 N 變大，必須設法讓實際通訊量遠低於這個上限——只在需要時通訊、只傳最有價值的資料。

四、多機器人 SLAM 的完整流程

圖 27-5　多機器人協作 SLAM 的處理流程

【各機器人機載】

每台獨立執行單機 SLAM（第 13–25 章）→ 各自的位姿圖與地圖

↓ 提取精簡表示（關鍵幀描述子、稀疏特徵）

【通訊】

依優先順序交換候選資訊（頻寬有限，只傳高價值的）

↓

【跨機器人閉環偵測】

地點辨識比對 → 幾何驗證 → 離群點剔除

↓ 取得跨機器人的相對變換

【地圖融合】

建立跨機器人邊 → 合併位姿圖

↓

【全域最佳化】

位姿圖優化（含穩健核函數）→ 全域一致的地圖

↓

【分發】把修正後的結果回傳各機器人

案例

案例 27-1　地下挑戰賽：多機器人 SLAM 的極限考場

第 1 章案例 1-2 介紹過 DARPA 地下挑戰賽，此處從多機器人的角度重看這場比賽。

比賽要求機器人團隊在人造隧道、地下城市與天然洞穴中自主探索、建圖並尋找目標物。

這個場域幾乎集合了本章所有挑戰的最惡劣版本：

107. 通訊極差：地下環境無線電訊號衰減嚴重，機器人往往在深入後就完全失聯——這正是 27.3 節「零星、間歇通訊」的極端情況。

108. 環境退化：隧道的幾何高度重複（第 17.3 節的結構退化），感知混淆風險極高，錯誤閉環的機率大增。

109. 異質團隊：參賽隊伍普遍組合輪型機器人、四足機器狗與無人機，各自適應不同的地形（27.1 節的能力互補）。

這場比賽直接催生了多個重要的系統。LAMP 與 LAMP 2.0 正是為了處理 DARPA 地下挑戰賽中的地下多機器人 SLAM 而發展；LAMP 2.0 引入以圖神經網路為基礎的集中式優先排序機制，預測每個跨機器人閉環候選經過位姿圖優化後的結果。

這個「用 AI 預測哪些閉環候選值得處理」的做法極具啟發性：當資源（通訊、運算）有限時，與其平均對待所有候選，不如用學習方法預測價值再分配資源。這與 27.2 節的優先排序思想相同，只是用了不同的技術手段。

對同學的啟示是：極端場景往往是技術突破的溫床。地下挑戰賽的參賽系統，如今已成為多機器人 SLAM 的重要基礎——困難的問題逼出了好的答案。

案例 27-2　通訊不可靠時，系統會壞到什麼程度

多機器人 SLAM 的論文，多半在通訊良好的條件下評測。但真實場域中，通訊往往時斷時續——這對系統的影響有多大？

有研究專門量化了這個問題，結果相當驚人：相對於可靠通訊的基準，Swarm-SLAM 的誤差增加為 5.08 倍、Kimera-Multi 為 6.17 倍。

換句話說，同一套系統在通訊不可靠時，誤差可能惡化五到六倍。這個數字提醒我們：實驗室的漂亮數據，未必能在真實場域重現。

這也解釋了為何近年的研究越來越重視通訊條件。第 27.4 節提過的「把通訊品質視為規劃資源」正是一個回應：與其被動接受通訊不良的後果，不如主動規劃路線以維持連線，或設計能在斷線時優雅降級的架構。

對做專題的同學而言，這是一個重要提醒：評測你的多機器人系統時，務必加入「通訊不良」的情境測試。只在理想條件下測試，會嚴重高估系統的實際能力。這與第 14.6 節「常見失敗案例」、第 26.5 節「挑戰測試」的精神一致——主動尋找失效邊界，是負責任的工程實踐。

案例 27-3　從模擬到現場：異質團隊的開發工具

多機器人系統的開發有一個現實困難：硬體成本。十台機器狗的價格對多數實驗室都是沉重負擔，而且調度十台機器人做實驗的人力與場地成本同樣可觀。

模擬因此成為多機器人研究的必要工具。近年已有專為異質多機器人設計的開源模擬環境出現。例如有開源模擬器與資料蒐集管線，建立在虛幻引擎 5 的模擬器之上，解決了先前框架的關鍵架構限制，以支援在大規模、照片級真實、動態環境中同時運作無人空中與地面載具；它引入了新的路徑點追蹤地面載具控制器以對應既有的無人機控制介面，並為異質平台提供共享的導航堆疊，涵蓋建圖、可通行性分析、規劃與控制。

這類工具對教學特別有價值：同學能在模擬中建立十台、二十台機器人的團隊，測試任務分配、地圖融合與通訊策略，完全不需要對應數量的實體硬體。等到演算法在模擬中驗證可行，再用兩三台實體機器人驗證關鍵環節即可。

這正是本書從第 8 章就建議的學習路徑：先模擬、再便宜硬體、後追求規模。多機器人系統把這個原則的價值放大到極致——沒有模擬，多機器人研究幾乎寸步難行。

實作活動

實作 27-1　多機器人協作建圖模擬

目的：透過模擬完整體驗多機器人 SLAM 的核心議題——地圖融合、通訊限制、任務分配與錯誤閉環的風險。本實作以模擬為主，不需多台實體機器人。

第一部分：兩機器人地圖融合（全體）

25. 沿用第 25 章實作 25-1 的模擬環境（或第 13 章的 2D 建圖模擬），建立兩台虛擬機器人，各自從房間的不同角落出發，以不同路線探索並建立自己的地圖。
26. 觀察問題：把兩張地圖直接疊在一起顯示，會發現它們完全對不上——因為各自以自己的起點為原點（27.1 節的座標系統一問題）。
27. 人工融合：假設你「知道」兩台機器人起點的相對位置，依公式一做座標變換，把 B 的地圖轉到 A 的座標系，再疊合顯示。觀察融合後的地圖是否完整。
28. 自動估計變換：讓兩台機器人的探索路線有部分重疊，用第 17 章的 ICP 對兩張地圖（或重疊區域的點）做配準，自動估出相對變換。與真值比較誤差。
29. 加入漂移：讓兩台機器人的位姿估計都帶有累積誤差，重新融合，觀察地圖如何錯位，並用位姿圖優化（第 12 章）修正。

第二部分：通訊限制實驗（全體或大學部）

30. 設定通訊範圍：規定兩台機器人只有在距離小於某個值時才能交換資料。模擬它們各自探索、偶爾相遇的過程。
31. 比較三種通訊條件：全時連線、只在相遇時通訊、以及隨機斷線（例如每步有 30% 機率無法通訊），比較最終地圖的品質差異。
32. 量化影響：計算三種條件下的地圖誤差，與案例 27-2 提到的「誤差增加五到六倍」對照，看看你的模擬是否呈現類似趨勢。
33. 資料量統計：記錄三種條件下的總通訊量，思考「通訊量」與「地圖品質」的取捨。

第三部分：任務分配與錯誤閉環（大學部）

34. 探索效率比較：讓 1 台、2 台、4 台機器人分別探索同一個房間直到全區覆蓋，記錄各自所需的步數。畫出「機器人數量 vs 完成時間」的曲線，觀察效益是否線性成長（提示：通常不會，因為會有重複探索）。

35. 任務分配策略：比較「隨機探索」與「依區域劃分」兩種策略的效率差異，對照 27.5 節的討論。

36. 錯誤閉環實驗：人為加入一條錯誤的跨機器人閉環邊（例如把 A 的某位姿錯誤地連到 B 的不相關位姿），執行位姿圖優化，觀察地圖如何被破壞。

37. 穩健化：在最佳化中加入穩健核函數（可用 scipy 的 soft_l1 等損失函數），重跑一次，觀察錯誤閉環的破壞是否被抑制。這直接驗證公式二中 ρ 的作用。

討論問題

11. 未做座標變換時，兩張地圖疊起來是什麼樣子？這說明了多機器人 SLAM 相對單機的什麼新問題？

12. ICP 自動估計的相對變換誤差有多大？若兩台機器人的探索路線幾乎沒有重疊，會發生什麼？

13. 通訊條件從「全時連線」變成「只在相遇時通訊」，地圖誤差增加了幾倍？與案例 27-2 的數據相比如何？

14. 機器人從 1 台增加到 4 台，完成時間縮短為幾分之一？為什麼不是四分之一？

15. （大學部）加入錯誤閉環後，地圖的哪些部分被破壞得最嚴重？穩健核函數把破壞抑制了多少？這對真實系統的設計有什麼意涵？

延伸挑戰（大學部）

其一：實作簡易的閉環優先排序——當通訊量有限，只能傳輸部分閉環候選時，設計一個評分機制（例如依據「這條閉環能連接多遠的兩段軌跡」或「涉及的機器人是否已有其他連結」），優先傳輸高分候選，比較它與隨機選擇的效果差異。這對應 27.2 節的核心思想。

其二：異質團隊模擬——加入一台「速度快但感測範圍小」與一台「速度慢但感測範圍大」

的機器人，設計任務分配策略以發揮各自優勢，並與同質團隊比較效率。這呼應 27.1 節的能力互補與第 26.7 節的地空協作。

延伸閱讀

15. Y. Tian et al., "Kimera-Multi: Robust, Distributed, Dense Metric-Semantic SLAM for Multi-Robot Systems," IEEE T-RO, 2022.（分散式多機器人語意 SLAM 的代表作，程式碼公開）
16. P.-Y. Lajoie and G. Beltrame, "Swarm-SLAM: Sparse Decentralized Collaborative SLAM Framework for Multi-Robot Systems," IEEE RA-L, 2024.（ROS 2 去中心化框架，含閉環優先排序技術）
17. Y. Chang et al., "LAMP 2.0: A Robust Multi-Robot SLAM System for Operation in Challenging Large-Scale Underground Environments," IEEE RA-L, 2022.（DARPA 地下挑戰賽的代表系統）
18. P. Schmuck and M. Chli, "CCM-SLAM: Robust and Efficient Centralized Collaborative Monocular SLAM," Journal of Field Robotics, 2019.（集中式協作 SLAM 的經典）
19. P.-Y. Lajoie et al., "DOOR-SLAM: Distributed, Online, and Outlier Resilient SLAM for Robotic Teams," IEEE RA-L, 2020.（分散式架構與離群點剔除）
20. awesome-Collaborative-SLAM 開源清單（GitHub）.（協作 SLAM 論文與程式碼的完整整理，研究入門的好起點）
21. S. Macenski et al., "Robot Operating System 2: Design, Architecture, and Uses in the Wild," Science Robotics, 2022.（ROS 2 的多機器人支援，第八篇實作的基礎）

小結

15. 多機器人 SLAM 的四項優勢為速度、容錯、視角互補與能力互補；三大新挑戰為座標系統一、資料關聯範圍擴大、以及通訊限制。其核心技術是單機技術（SLAM、地點辨識、位姿圖優化、離群點剔除）在多座標系與多資料來源上的擴充。

16. 地圖融合分三步驟：偵測相遇、估計相對變換、融合與最佳化；最大風險是錯誤的跨機器人閉環——候選空間更大、感知混淆更嚴重、後果更嚴重，因此穩健的離群點剔除是必備元件，而閉環候選的優先排序能在有限頻寬下獲得最大效益。

17. 三種架構各有取捨：集中式融合簡單但有通訊瓶頸與單點故障、無法擴展到大型團隊；分散式與去中心化執行點對點位姿圖優化，擴展性佳且對通訊中斷有韌性，代價是同步複雜度與機載運算負擔；只依賴零星間歇通訊的去中心化解法更適合大範圍場景。

18. 機載、邊緣、雲端三層依即時性、資料量與隱私分工；邊緣輔助的做法是以機載視覺里程計維持自主性、把繁重運算卸載到伺服器；近年更把通訊品質視為規劃資源，使導航斷崖成為可管理的降級事件而非硬性極限。

19. 多機器狗校園巡檢採機載、邊緣、雲端三層架構；任務分配可依區域、依邊界動態或依能力；實務課題包含起始座標對齊（可用共同充電站或 AprilTag 簡化）、通訊覆蓋、互相避讓與電池輪替。

20. AI Agent 在多機系統中扮演任務規劃者、異常判讀者與協調仲裁者；共享語意而非只共享幾何能讓團隊對環境的理解保持一致，但須留意延遲、幻覺與通訊依賴三項限制。

21. 四項工程課題：隊形需兼顧避撞、通訊鏈與覆蓋效率；通訊須面對頻寬限制與間歇連線；資料同步含時間、座標與版本一致；安全涵蓋通訊加密、資料隱私、實體安全與故障隔離。

承先啟後

本章把「多來幾台」這件事的三個關鍵參數量化了：協同探索的加速比在六台左右飽和於 11.3 倍，而且共享地圖與否的差距極大——八台時共享讓完成時間縮短 83%；地圖融合的正面價值是把 ATE 從 0.576 公尺降到 0.119 公尺，但只要混入一條錯誤的跨機邊就會惡化到 1.238 公尺，比完全不融合還糟兩倍，而穩健核能把它穩定在 0.47 公尺；頻寬方面，依代數連通度排序後只需十條跨機閉環，就能達到隨機挑選三十條的效果。

第七篇到此完成。從第 25 章的毫瓦級微型平台、第 26 章的無人機導航，到本章的群體協作，呈現了一條清晰的線索：當單一機器人的能力受限時，答案不是把它做得更大更強，而是讓它更小、更省電，然後用更多台協同工作——這與螞蟻和蜜蜂的策略不謀而合。

下一篇，我們回到最實際的層面：如何把這一切真正跑起來。第八篇「ROS / Python / C++ 實作」將帶同學建置開發環境、實作 2D 光達建圖與導航、執行視覺 SLAM，並完成完整的專題與評量。理論走到這裡，是時候讓機器人真的動起來了。